

The Globe:

OPOSICIONES

TAI · BLOQUE 3

BLOQUE 3 · TEMA 05

T05

DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS. ELEMENTOS Y COMPONENTES SOFTWARE: OBJETOS, CLASES, HERENCIA, MÉTODOS, SOBRECARGA. VENTAJAS E INCONVENIENTES. PATRONES DE DISEÑO Y LENGUAJE DE MODELADO UNIFICADO (UML)

FORMADOR
Pablo Arellano

1. DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS

El proceso de desarrollo orientado a objetos se compone de:

1. **Especificación:** definición de qué va a hacer un programa. En esta fase se debe pensar detenidamente en la funcionalidad que se quiere del sistema, en sus casos de uso, etc.
2. **Diseño:** basándose en la información anterior, se plantea una solución que determina:
 - qué clases van a hacer falta para representar el problema.
 - qué comportamiento (operaciones) van a tener esas clases.
 - cómo se relacionan entre ellas.

En esta fase se utilizan muchas veces diagramas.

3. **Codificación:** también llamada implementación. Una vez está claro cómo se plantea la solución, hay que codificar las clases. Para ello, hay que basarse en los principios de:
 - Claridad: el objetivo es que el código sea fácil de entender por el diseñador y por otras personas, para facilitar su depuración, mantenimiento, etc. Para ello, se cuenta con reglas de estilo, de nombrado, de indentación, de documentación, etc.
 - Eficiencia: se trata de buscar soluciones que, para obtener el mismo resultado, consumar el menor número de recursos posibles. Por recursos se entiende fundamentalmente la memoria y el tiempo de CPU.
4. **Prueba:** se compila y ejecuta el programa para ver si cumple la especificación. Si no lo hace, puede haber errores de codificación o de diseño y hay que volver hacia atrás.
 - Diseñar y codificar las pruebas a la vez que se diseña y codifica el programa.
 - Probar el mayor número de casos posibles.
 - Probar a la vez que se va codificando, de menor a mayor:
 - o Cada operación de una clase.
 - o La clase completa.
 - o La integración de varias clases.
5. **Mantenimiento:** una vez se pone el programa en funcionamiento, no termina su ciclo de vida:
 - Se pueden detectar nuevos errores.
 - Se puede querer cambiar o ampliar su funcionalidad.

Por ello es muy importante que el programa sea claro, legible y modular:

- Los cambios no afectarán a todo el programa.
- La persona encargada del mantenimiento probablemente no sea la misma que lo programó.

Las **características** más importantes de la programación orientada a objetos son las siguientes (vistas en el tema B3T3):

- **ABSTRACCIÓN**: denota las características esenciales de un objeto, donde se captura su comportamiento.
- **ENCAPSULAMIENTO**: hace referencia a la reunión de todos los elementos que pueden considerarse pertenecientes a una misma entidad. Ello supone el aumento de la cohesión de los componentes. No confundir con el principio de ocultación.
- **PRINCIPIO DE OCULTACIÓN**: la estructura interna permanece oculta, tanto para el usuario como para otros objetos diferentes. La información contenida en el objeto será accesible sólo a través de la ejecución de los métodos adecuados creándose una interfaz para la comunicación con el mundo exterior.
- **POLIMORFISMO**: comportamientos diferentes, asociados a objetos distintos, pueden compartir el mismo nombre; al llamarlos por ese nombre se utilizará el comportamiento correspondiente al objeto que se esté usando. Dicho de otro modo, las referencias y las colecciones de objetos pueden contener objetos de diferentes tipos y la invocación de un comportamiento en una referencia producirá el comportamiento correcto para el tipo real del objeto referenciado. Cuando esto ocurre en “tiempo de ejecución”, esta última característica se llama “asignación tardía” o “asignación dinámica”. Algunos lenguajes proporcionan medios más estáticos (en “tiempo de compilación”) de polimorfismo.
- **HERENCIA**: la herencia es la propiedad por la cual una clase asume como propios todos los atributos y métodos definidos por otra. La que hereda se denomina «subclase» o «clase derivada». La que transmite la herencia recibe el nombre de «superclase» o «clase base».

2. PRINCIPIOS DE GENERALES DE PROGRAMACIÓN

1. Principios SOLID

Son cinco principios de diseño destinados a hacer que los diseños de software sean más comprensibles, flexibles y mantenibles. Los principios son un subconjunto de muchos principios promovidos por el ingeniero e instructor de software estadounidense Robert C. Martin. Aunque se aplican a cualquier diseño orientado a objetos, los principios SOLID también pueden formar una filosofía central para metodologías como el desarrollo ágil o el desarrollo de software adaptativo.

Los principios comprendidos en SOLID son:

- **S**: Single responsibility. Principio de responsabilidad única.
- **O**: Open/closed. Principio de abierto/cerrado.
- **L**: Liskov substitution. Principio de sustitución de Liskov.
- **I**: Interface segregation. Principio de segregación de interfaz.
- **D**: Dependency inversion. Principio de inversión de dependencia.

Algunas de las ventajas de aplicar estos principios son:

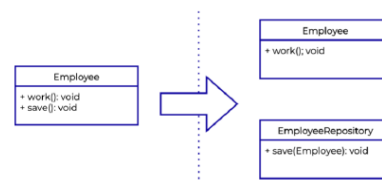
- Facilitar el mantenimiento del código.
- Reducir la complejidad de añadir nuevas funcionalidades.
- Aumentar la reusabilidad de piezas y componentes.
- Mejorar la calidad del código y su comprensión.

Single responsibility (PRINCIPIO DE RESPONSABILIDAD ÚNICA)

Establece que un módulo de software debe tener una y solo una razón para cambiar. Esta razón para cambiar es lo que se entiende por responsabilidad. Cada clase debería tener responsabilidad sobre una única funcionalidad del software.

Este principio está estrechamente relacionado con los conceptos de acoplamiento y cohesión. Queremos aumentar la cohesión entre las cosas que cambian por las mismas razones y disminuir el acoplamiento entre las cosas que cambian por diferentes razones. Este principio trata sobre **limitar el impacto de un cambio**.

Ejemplo: la clase Employee tiene 2 responsabilidades, la propia de un empleado que es trabajar y la de persistir su estado. Conviene separar ambas responsabilidades en 2 clases.



Open/closed (ABIERTO A LA EXTENSIÓN, CERRADO A LA MODIFICACIÓN)

Este principio nos dice que los módulos de software deben ser abiertos para su extensión pero cerrados para su modificación:

- **Abierto** para su extensión es que el comportamiento del módulo puede extenderse.
- **Cerrado** para su modificación es que su interface es estable y bien definida.

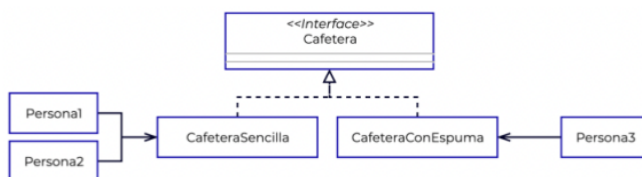
Este principio pretende minimizar el efecto cascada que puede suceder cuando se cambia el comportamiento de una clase si existen clientes que dependan de ella.

El polimorfismo es el concepto que une ambos conceptos.

Ejemplo: partiendo del siguiente diagrama se requiere una funcionalidad distinta para cafetera (cambio en su comportamiento).



En lugar de modificar el comportamiento de la cafetera actual, se aplica el principio open/closed, es decir, se cierra la clase CafeteraSencilla impidiendo modificaciones y se crea una interfaz para abrirla a extensiones:

**Liskov substitution (OBJETOS DE SUBCLASES SE COMPORTAN IGUAL QUE LOS DE LA SUPERCLASE)**

La sustitución de Liskov nos dice que los objetos de un programa deberían ser reemplazables por instancias de sus subtipos sin alterar el correcto funcionamiento del programa.

Una clase que hereda de otra debe poder usarse como su padre sin necesidad de conocer las diferencias entre ellas.

Este principio nos ayuda a utilizar la herencia de forma correcta ya que los objetos de las subclases se deben comportar de igual forma que los de la superclase.

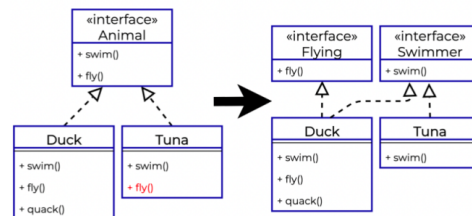
Interface segregation (MEJOR INTERFACES ESPECÍFICAS A INTERFACES GENERALES)

El principio de segregación de interfaces establece que muchas interfaces cliente específicas son mejores que una interfaz de propósito general. Cuando los clientes son forzados a utilizar interfaces que no usan por completo, están sujetos a cambios de esa interfaz. Esto al final resulta en un acoplamiento innecesario entre los clientes.

Esto es, cuando un cliente depende de una clase que implementa una interfaz, cuya funcionalidad este cliente no usa pero que otros clientes si, este cliente estará siendo afectado por los cambios que fueren otros clientes en la clase en cuestión.

En definitiva, ninguna clase debería depender de métodos que no usa.

Ejemplo: la clase Tuna tiene el método `fly()` que no va a implementar. Conviene separar la interfaz `Animal` en 2.

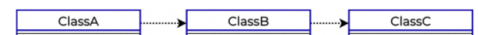


Dependency inversión (DEPENDER DE ABSTRACCIONES, NO DE IMPLEMENTACIONES)

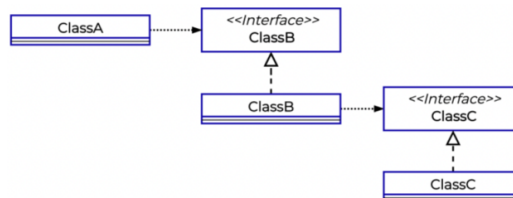
El principio de inversión de dependencia propugna que las entidades de software deben depender de abstracciones, no de implementaciones.

A su vez, los módulos de alto nivel no deben depender de módulos de bajo nivel, ambos deben depender de abstracciones. El objetivo de este principio es evitar la dependencia de concreciones para minimizar el grado de acoplamiento entre módulos.

Ejemplo: las capas de alto nivel (`ClassA`) suelen consumir las de bajo nivel (`ClassC`), generando un fuerte acoplamiento.



Para evitar este problema, se introduce una capa de abstracción que permite reutilizar las capas de mayor nivel:



2. Don't Repeat Yourself (DRY)

El principio "No te repitas" consiste en evitar duplicaciones lógicas en el software.

Cada pieza de funcionalidad debe tener una identidad única, no ambigua y representativa dentro del sistema.

Según este principio toda pieza de funcionalidad nunca debería estar duplicada ya que esta duplicidad incrementa la dificultad en los cambios y crear inconsistencias.

Pieza de funcionalidad se refiere a una función lógica, no a un código.

Ejemplo: no conviene disponer de 3 métodos distintos para abrir una conexión a una base de datos. Son 3 métodos con código distinto para una misma función lógica.

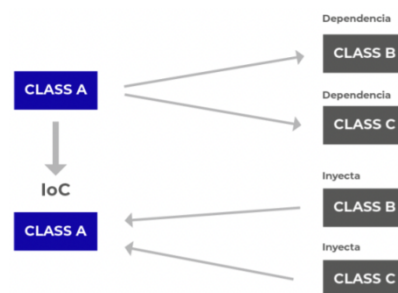
Las ventajas de este principio es que la mantenibilidad del código, la reducción del tamaño del mismo y el ahorro de tiempo.

3. Inversion of Control (IoC)

Consiste en delegar en un tercero diferentes tipos de flujos de control para lograr un bajo acoplamiento. Esto incluye el control sobre el flujo de una aplicación y el control sobre el flujo de la creación de un objeto o la creación y vinculación de objetos dependientes.

El principio de IoC ayuda a aumentar la modularidad del programa y al diseño de clases con bajo acoplamiento, lo que mejora la testeabilidad, mantenibles y extensibles.

Ejemplo: se cambian las dependencias entre clases por inyecciones.



4. Your Aren't Gona Need It (YAGNI)

El principio “No vas a necesitar eso” indica que no se deben agregar funcionalidades extras hasta que no sea necesario.

La tentación de escribir código que no es necesario pero que puede serlo en un futuro, supone el desperdicio del tiempo que se destinaría a funcionalidades básicas o que cuando se requieran las nuevas funcionalidades, estas no funcionen correctamente.



5. Keep It Simple, Stupid (KISS)

El principio “Mantelo simple, estúpido” establece la necesidad de minimizar los errores tratando de realizar las tareas de forma efectiva y eficiente complicándose lo mínimo posible. Buscar la simplicidad y huir de la complejidad innecesaria.

6. Ley de Demeter (LoD)

La idea principal es que un objeto no tiene porqué conocer la estructura interna de los objetos con los que colabora. Lo que se quiere evitar es el código con una estructura similar a la siguiente:

```
object.getX().getY().getZ().doSomething()
```

Este tipo de estructuras conlleva un alto acoplamiento y que cualquier cambio en las clases afecta al resto.

Para lograrlo, la LoD establece que un método de un objeto solo debe interactuar con métodos del propio objeto, con los argumentos que recibe, con cualquier objeto creado dentro del método o con cualquier campo directo dentro del propio objeto.

3. ELEMENTOS Y COMPONENTES SOFTWARE

Elementos:

- Clases.
- Objetos.
- Métodos.
- Atributos.
- Mensajes.
- Herencia.
- Polimorfismo.
- Sobrecarga.

1. Clases

Una **clase** es el conjunto de métodos y datos que resumen las características comunes de todos los objetos que la componen. Una clase está compuesta por un conjunto de objetos diferentes que pueden actuar de formas muy similares. Las clases existen en tiempo de compilación.

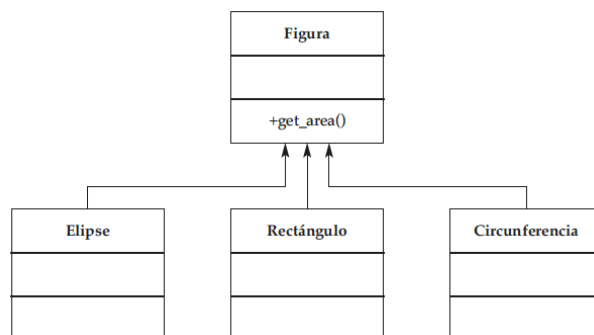
Las clases son las “plantillas” desde las que se pueden crear o instanciar múltiples objetos del mismo tipo. O lo que es lo mismo, representan la abstracción de las características comunes de un tipo determinado de objetos. Los componentes de las clases son:

- **Atributos:** elementos que mantienen el estado del objeto. Suelen ser privados, es decir, solo se pueden acceder a ellos desde el interior de la clase.
- **Métodos:** mensaje para realizar alguna acción en un objeto. Suelen ser públicos. Permiten desde el exterior modificar de forma controlada los atributos de un objeto.

Cada uno de los objetos individuales pertenecientes a una clase se denomina **instancia**. Desde el punto de vista de la programación, se puede considerar a una clase como un tipo de datos de los disponibles en el sistema.

Una clase que no tenga instancias se denomina **clase abstracta**. Una clase abstracta puede servir para declarar las propiedades de un conjunto determinado de clases pero que a ese nivel de abstracción no se pueden concretar.

Ejemplo: el método `obtener_area()` puede ser método común a todas las figuras por lo que se podrá declarar a nivel de la clase `Figura` pero no se podrá definir porque no existe un método común para obtener el área de todos los tipos de figuras. Se podrá definir al nivel de `Elipse`, `Circunferencia` o de `Rectángulo`.



2. Objetos

Los objetos son módulos que contienen los datos y las instrucciones que manipulan esos datos. Un programa OO consta sólo de objetos que contienen tanto los métodos como la estructura de datos. Los objetos existen en tiempo de ejecución.

Los objetos son entidades que tienen un determinado estado, comportamiento (método) e identidad:

- El **ESTADO** está compuesto de datos o informaciones.
- El **COMPORTAMIENTO** está definido por los métodos o mensajes a los que sabe responder.
- La **IDENTIDAD** es una propiedad de un objeto que lo diferencia del resto.

Los objetos, por tanto, son entidades que tienen atributos (datos) y formas de comportamiento (métodos) particulares. Dentro de un sistema orientado a objetos se representarán todos aquellos relevantes para el Universo de Discurso sobre el que trate el sistema. En este sentido, el diseño orientado a objetos se centrará sobre cualquier cosa real o abstracta sobre la que se tiene una percepción de su individualidad.

Clase	Objetos		
Persona	Persona	Persona	Persona
nombre	José	Laura	Cristina
edad	23	37	19
profesión	carpintero	administrativa	estudiante
dameNombre	dameNombre	dameNombre	dameNombre
dameEdad	dameEdad	dameEdad	dameEdad
dameProfesión	dameProfesión	dameProfesión	dameProfesión

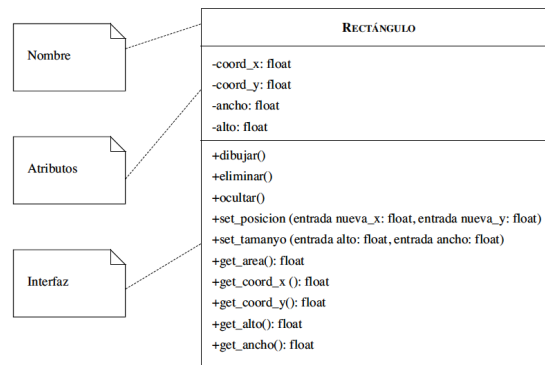
3. Métodos

El conjunto de procedimientos compartidos que contienen los objetos se denomina métodos. Los métodos definen la forma en la que los datos contenidos en los objetos son manipulados.

Los métodos de un objeto se invocan exclusivamente con el mensaje adecuado, y al ser invocado un método de un objeto, sólo se referirá a la estructura de datos de ese objeto y no a la de otros, aunque sean de la misma clase.

La interfaz de la clase estará definida por el conjunto de métodos que soporta y los mensajes que es capaz de tratar.

Ejemplo: definición de una clase con sus atributos y métodos.



Métodos a destacar:

- **Constructor**: crea un objeto e inicializa sus atributos. Pueden existir varios constructores para una misma clase. No definen ningún tipo de dato de retorno.
- **Destructor**: se invoca implícitamente y destruye un objeto. Según el lenguaje no es necesario siquiera definirlo.
- **Getter**: devuelve el valor de un atributo/propiedad de un objeto.
- **Setters**: modifica el valor de un atributo/propiedad de un objeto.

4. Atributos

Los atributos son las propiedades que encapsula una clase y que definen el estado de un objeto. El tipo de datos de un atributo puede ser un tipo de datos básico o una clase que se haya definido previamente.

Podemos distinguir varios tipos de acceso a los atributos:

- **Público (+)**: se puede acceder desde cualquier clase.
- **Privado (-)**: solo se puede acceder desde los métodos de la clase donde están definidos.
- **Protegido (#)**: se pueden acceder desde la propia clase y desde clases hijas.

5. Mensajes

Los objetos tienen la posibilidad de actuar, de comunicarse entre sí. Dicha comunicación sucede cuando un objeto recibe un mensaje, que no es más que una solicitud que le pide que se comporte de alguna forma determinada.

El mensaje contiene el nombre del objeto al que va dirigido, el nombre de una operación y, en ocasiones, un grupo de parámetros.

Al ejecutar un programa OO, los objetos reciben, interpretan y responden a estos mensajes procedentes de otros objetos. El conjunto de mensajes al que puede responder un objeto se denomina protocolo del objeto.

6. Herencia

La herencia es el mecanismo mediante el cual una clase (**subclase**) adquiere las propiedades de otra clase jerárquicamente superior (**superclase** o clase base). La herencia proporciona el mecanismo para compartir automáticamente métodos y datos entre clases, subclases y objetos, y puede ser simple o múltiple, dependiendo si una subclase hereda los datos y métodos de una sola clase o de más de una (dependiendo del lenguaje concreto).

Una clase se puede definir de manera muy amplia y después refinarla en sucesivas subclases. Cada subclase incorpora o «hereda» todas las propiedades de su superclase y añade sus propiedades únicas. Las propiedades de la superclase no necesitan repetirse en cada subclase, por tanto, la posibilidad de reunir las propiedades comunes de varias clases en una superclase común y heredarlas de la misma, reduce en gran medida la repetición en el diseño y la programación y es una de las principales ventajas de la OO.

La herencia proporciona relaciones entre clases del tipo “**es-un**”:

- La herencia de implementación implica que la clase hija hereda la implementación de métodos de la clase padre. Si se sobrescribe el código del método heredado en la clase hija, se denomina **redefinición**.
- La herencia de interfaz (interface) implica que la clase hija hereda la interfaz (pero no la implementación de las operaciones).
- La herencia de clase combina la herencia de implementación y de interfaz.

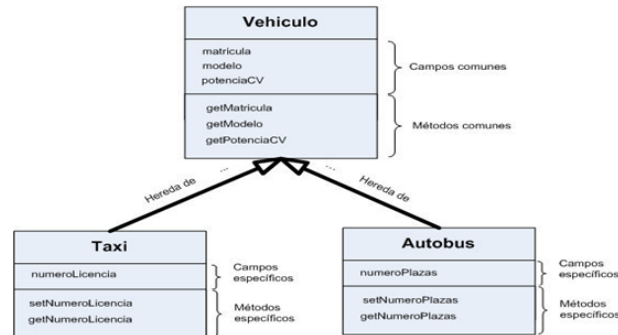
Algunos lenguajes tienen palabras clave para distinguir entre herencia de interfaz y de clase.

La herencia puede ser:

- **Simple**: una clase solo puede heredar de una superclase (Java).
- **Múltiple**: una clase puede heredar de más de una superclase (C++). Problema: ambigüedad, varios métodos con la misma signatura (produciría colisiones).

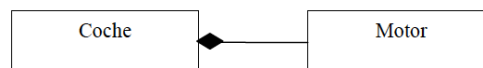
Existen un tipo de clases denominadas **clases abstractas** las cuales declaran métodos que no implementan. Una clase será abstracta cuando al menos uno de sus métodos no esté implementado. Una clase abstracta no puede ser instanciada.

Ejemplo: Herencia.



7. Composición

La composición es un mecanismo por el que se pueden crear clases complejas agregando objetos de clases ya existentes. Su relación se le conoce como “**tiene-un**”. Permite reutilizar código.

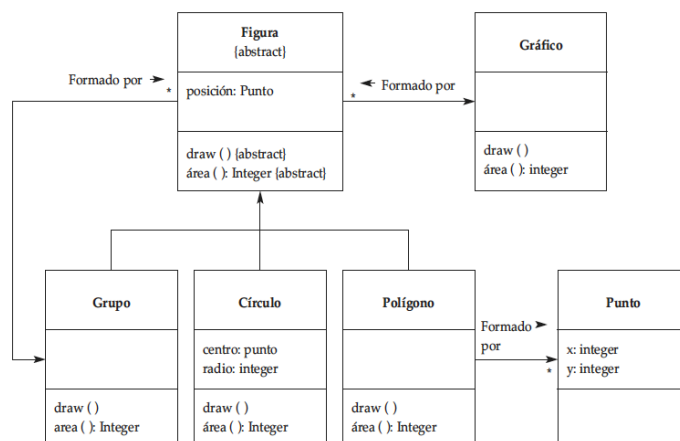


8. Polimorfismo

El polimorfismo es la propiedad por la cual un mismo mensaje puede originar conductas diferentes al ser recibido por objetos diferentes. Es decir, la misma operación puede comportarse de manera diferente para clases diferentes.

El polimorfismo es consecuencia de la herencia. El mismo método está declarado con el mismo nombre en diferentes clases (jerarquía). Esto requiere que el lenguaje disponga de enlace dinámico mediante la cual se ejecute el método correcto en tiempo de ejecución (no estáticamente).

Ejemplo: el método `área()` se define como abstracto (sin implementación) en la clase `Figura`. Las subclases de `Figura` heredan dicho método. Es obvio que el cálculo del área depende del tipo de figura concreta. Ahora creamos un objeto de la clase `Figura`. Este objeto puede ser un `Grupo`, un `Círculo` o un `Polígono`. En función del tipo concreto de objeto se ejecutará el método `área` de una clase u otra. Esto es el polimorfismo.



9. Sobrecarga

Los métodos se distinguen por su *signatura*, y la signatura está formada tanto por el nombre como por el número y tipo de los argumentos que recibe el método. Recuerda que el tipo de retorno de un método no forma parte de su signatura, y por lo tanto no sirve para diferenciar métodos.

Ejemplo: si partimos de la clase Punto3D (mostrada parcialmente), podemos decir que el constructor está sobrecargado 3 veces. Ahora bien:

```

public class Punto3D {
    private int x;
    private int y;
    private int z;
    public Punto3D() {
        x = y = z = 0;
    }
    public Punto3D(int x) {
        this.x = x;
    }
    public Punto3D(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

```

```

new Punto3D(1); //¿Qué atributo estamos iniciando, x, y o z?
new Punto3D(1, 1); //¿Qué atributo NO estamos iniciando?

```

El lenguaje ha de seleccionar el método al que se invoca según su signatura. La sobrecarga nos permite seleccionar el método adecuado.

ATENCIÓN

No confundir sobrecarga con sobreescritura (@Override).

Sobrecargar significa definir nuevos métodos dentro una misma clase.

Sobrescribir significa ocultar un método con una nueva definición de ese mismo método en una clase hija.

La sobrecarga no implica herencia, la sobreescritura sí.

Por último, algunos lenguajes permiten la **sobrecarga de operadores** (=, +, -, etc.), es decir, admiten definir el comportamiento del operador para un objeto que previamente han sido definidos para la clase a la que pertenece el objeto. C++ contempla la sobrecarga de operadores.

4. VENTAJAS E INCONVENIENTES ORIENTACIÓN A OBJETOS

Ventajas de la orientación a objetos:

- Reutilización de código y diseños (**reusabilidad**), lo que aumenta la productividad.
- Los sistemas orientados a objetos son generalmente más pequeños que su equivalente no orientado a objetos, lo que supone **menos código** y más reutilización.
- Permite la **interoperabilidad** entre aplicaciones, aislando las dependencias de las plataformas.
- **Facilita el mantenimiento** y extensión (**extensibilidad**) de las aplicaciones, dado que se tiene la posibilidad de ampliar la funcionalidad de la aplicación de manera sencilla.
- Suministra modelos similares a los del mundo real.
- Facilita el desarrollo de sistemas complejos, generando sistemas más **preparados al cambio**.
- Válido para aplicaciones de pequeño y gran tamaño.
- **Encapsulación** y **modularidad** porque el sistema se puede descomponer en objetos con unas responsabilidades claramente especificadas. Por lo tanto, los diseños serán más proclives a tener un acoplamiento bajo y una cohesión alta. El acoplamiento es una medida externa a las clases y mide el grado de interdependencia entre módulos. Se debe conseguir que los módulos sean lo más independientes entre sí. En principio, el propio diseño de clases debería favorecer esta medida. La cohesión es una medida interna a las clases y mide la relación existente entre los elementos de un módulo. Un módulo con alta cohesión realiza una tarea concreta y sencilla. En principio, el comportamiento de la clase debería estar enfocado al estado de datos que debe mantener.
- El diseño y análisis OO es más **natural** (modelado cercano a la realidad) que el paradigma estructurado.

Inconvenientes de la orientación a objetos:

- El uso de un lenguaje orientado a objetos no evitará malos análisis, diseños o programas. La POO ofrece herramientas para minimizar esto pero estas tareas son netamente responsabilidad de los desarrolladores.
- El equipo de trabajo necesita una preparación específica, tanto en el paradigma de OO como en el lenguaje y herramientas que se vayan a usar.
- La curva de aprendizaje de las múltiples librerías de componentes y de las múltiples herramientas así como de las técnicas de Ingeniería del Software necesarias para usar la OO exitosamente es mayor que la simple técnica de codificar todo lo que sea necesario sin reconocer lo existente.

- Los lenguajes OO puros son poco eficientes y necesitan de arquitecturas hardware relativamente potentes para funcionar correctamente. Los híbridos (como C++) son mucho más eficientes pero pueden comprometer los principios de la OO.
- La reusabilidad se ve seriamente comprometida si no hay una apuesta clara por la calidad de los paquetes de componentes desarrollados para ser reutilizados. El control de la calidad externa de los componentes (su funcionalidad) y la calidad interna (la manera de programarlos) será necesaria a lo largo de todo su ciclo de vida.

5. PATRONES DE DISEÑO

Un **patrón de diseño** es una solución a un problema de diseño cuya efectividad ha sido comprobada por haber sido empleada para resolver problemas similares en ocasiones anteriores.

Los patrones de diseño, según **The Gang of Four (GOF)**, describen soluciones simples y elegantes a problemas específicos en el diseño de software orientado a objetos.

Una característica fundamental es que deben ser **reusables**, lo que significa que sean aplicables a diferentes problemas de diseño en distintas circunstancias. Los patrones de diseño pretenden:

- Proporcionar catálogos de elementos reusables en el diseño de sistemas software.
- Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- Formalizar un vocabulario común entre diseñadores.
- Estandarizar el modo en que se realiza el diseño. Facilitar el aprendizaje de las nuevas generaciones de diseñadores condensando conocimiento ya existente.

Según el GOF, los patrones de diseño se agrupan en:

- **Patrones de CREACIÓN** (o creacionales): definen cómo puede crearse un objeto aislando los detalles de la creación del objeto, de forma que su código no dependa de los tipos de objeto que hay y, por lo tanto, no deba ser modificado al añadir un nuevo tipo de objeto.
Patrones:
 - o Abstract Factory (Factoría abstracta).
 - o Builder (Constructor).
 - o Factory Method (Factoría).
 - o Prototype (Prototipo).
 - o Singleton (Instancia única).
 - o Model View Controller (Modelo Vista Controlador).
- **Patrones ESTRUCTURALES**: tratan la manera en que los objetos se conectan con otros objetos, los combinan y forman estructuras mayores, asegurando estabilidad en las conexiones ya que los cambios del sistema no requieren cambiar esas conexiones.
Patrones:
 - o Adapter o Wrapper (Adaptador o Envoltorio).
 - o Bridge (Puente).
 - o Composite (Objeto compuesto).
 - o Decorator (Decorador).

- Facade (Fachada).
- Flyweight (Peso ligero).
- Proxy.
- **Patrones de COMPORTAMIENTO:** tratan a los objetos que manejan tipos particulares de acciones y llamadas entre los diferentes objetos dentro de un programa. Éstos encapsulan procesos que debe ejecutarse dentro de la funcionalidad de la aplicación, como interpretar un lenguaje, completar una petición, moverse a través de una secuencia o implementar un algoritmo. Patrones:
 - Chain of Responsibility (Cadena de responsabilidades).
 - Command (Comando).
 - Interpreter (Intérprete).
 - Iterator (Iterador).
 - Mediator (Mediador).
 - Memento (Recuerdo).
 - Observer (Observador).
 - State (Estado).
 - Strategy (Estrategia).
 - Template Method (Método plantilla).
 - Visitor (Visitante).

Por otro lado, los patrones **GRASP** (General Responsibility Assignment Software Patterns) son un conjunto de 9 patrones de diseño que se utilizan para asignar responsabilidades a clases y objetos en el diseño de software. Estos patrones proporcionan pautas para asignar responsabilidades de manera efectiva, ayudando a mejorar la modularidad y la flexibilidad del sistema.

Los patrones GRASP son:

- | | |
|----------------------|--------------------------|
| - Experto. | - Fabricación pura. |
| - Creador. | - Polimorfismo. |
| - Controlador. | - Indirección. |
| - Bajo acoplamiento. | - No hable con extraños. |
| - Alta cohesión. | |

1. Patrones creacionales



Abstract Factory (Factoría abstracta)

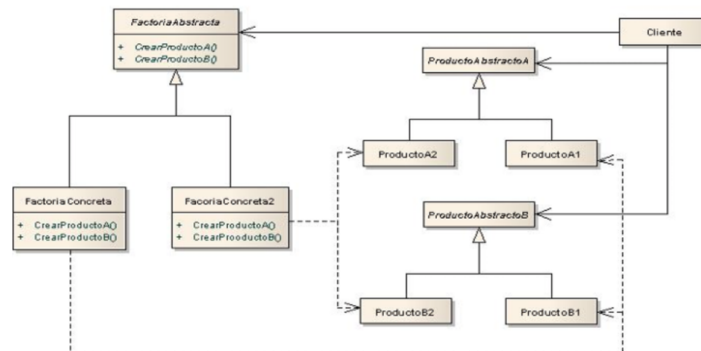
Es un patrón que define una interfaz para crear familias de objetos relacionados o dependientes entre sí sin especificar las clases concretas.

Motivación

Imaginemos un conjunto de herramientas de IU que soportan varios estándares de representación. En función de cada estándar, el comportamiento y la representación de los elementos de la IU varía (scrolls, menu bar...). El cliente no debe cambiar porque cambie la IU. El patrón Factoría Abstracta proporciona solución a esta problemática.

Aplicabilidad

- Un sistema debe de ser independiente de cómo se crean, componen y representan sus productos.
- Un sistema debe configurarse con una de entre varias familias de productos.
- Una familia de productos relacionados están hechos para usarse juntos y se necesita cumplir esa restricción.
- Se desea ofrecer una biblioteca de clases-producto revelando sus interfaces pero no sus implementaciones.



Participantes

- **Cliente:** la clase que llamará a la factoría adecuada ya que necesita crear uno de los objetos que provee la factoría. Es decir, el Cliente lo que quiere es obtener una instancia de alguno de los productos (ProductoA, ProductoB).
- **FactoríaAbstracta:** es la definición de las interfaces de las factorías. Debe de proveer un método para la obtención de cada objeto que pueda crear (crearProductoA() y crearProductoB()).
- **FactoríasConcretas:** estas son las diferentes familias de productos. Provee de la instancia concreta de la familia que se encarga de crear. De esta forma podemos tener una factoría que cree los elementos gráficos para Windows y otra que los cree para Linux, pudiendo añadir fácilmente (creando una nueva FactoríaConcreta) otra que los cree para Mac, por ejemplo.

- **ProductoAbstracto:** definición de las interfaces para la familia de productos genéricos. En el diagrama son ProductoA y ProductoB. En un ejemplo de interfaces gráficas podrían ser todos los elementos: Botón, Ventana, Cuadro de Texto, Combo, etc. El Cliente trabajará directamente sobre esta interfaz, que será implementada por los diferentes ProductosConcretos.
- **ProductoConcreto:** implementación de los diferentes productos. Podría ser por ejemplo BotónWindows y BotónLinux. Como ambos implementan Botón el Cliente no sabrá si está en Windows o Linux, puesto que trabajará directamente sobre la superclase o interfaz.



Builder (Constructor)

El patrón es usado para permitir la creación de una variedad de objetos complejos desde un objeto fuente (Producto). El objeto fuente se compone de una variedad de partes que contribuyen individualmente a la creación de cada objeto complejo a través de un conjunto de llamadas a interfaces comunes de la clase ConstructorAbstracto.

Motivación

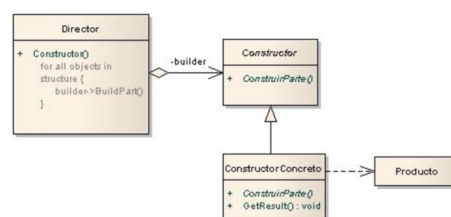
Imaginemos el caso de un lector de textos. A menudo, el lector tendrá que interpretar y convertir un tipo de documentos a otro tipo de lenguaje. El problema radica en que no sabemos muy bien cuantos tipos de lenguaje tendrá que estar preparado para interpretar. Se convierte en necesario crear una estructura que permita añadir una nueva especificación de un lenguaje sin modificar el lector. El patrón Constructor ofrece una solución para ello. Si imaginamos el lector como una clase que realiza un análisis de la información y que se lo pasa a una nueva subclase que realiza la conversión, esta subclase es la que irá añadiendo las especializaciones a nuevos formatos.

Aplicabilidad

- Que el algoritmo para la creación de objetos complejos sea independiente de las partes que construyen el objeto y cómo son ensambladas.
- Que el proceso de construcción pueda tener diferentes representaciones para el objeto que está construido.

Participantes

- **Constructor:** especifica una interfaz abstracta para construir partes del objeto producto.
- **ConstructorConcreto:** implementa la interfaz de Constructor, construyendo y ensamblando las partes del producto.
- **Director:** construye un objeto a través de la interfaz Constructor.
- **Producto:** representa el objeto complejo bajo construcción.





Factory Method (Factoría)

Centraliza en una clase constructora la creación de objetos de un subtipo de un tipo determinado, ocultando al usuario la casuística para elegir el subtipo que crear.

Motivación

Si pensamos en un framework, rápidamente identificamos que usa clases abstractas para la definición y mantenimiento de las relaciones entre objetos. A menudo es el framework el responsable de la creación de esos objetos. Este framework para aplicaciones puede presentar multitud de documentación al usuario. Una aplicación no puede prever que tipo de documentación necesita. El framework tiene que inicializar clases pero solo conoce las clases abstractas.

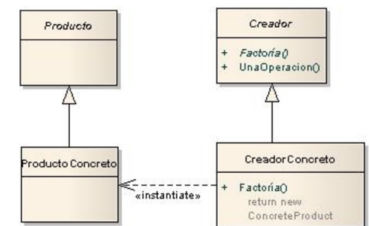
El patrón Factoría encapsula este conocimiento y lo saca fuera del framework, permitiendo que, mediante nuevas clases, se pueda identificar cuál es el documento asociado.

Aplicabilidad

- Una clase de objetos no puede prever la clase de objetos que tiene que crear.
- Las subclases son las que especifiquen los objetos que se crean.
- Las clases delegan la responsabilidad en una entre varias clases auxiliares.

Participantes

- **Producto**: define la interfaz de los objetos que la Factoría crea.
- **ProductoConcreto**: define la interfaz del Producto.
- **Creator**: declara el método `Factoría` que devuelve un objeto de tipo `Producto`.
- **CreatorConcreto**: sobrescribe el método `Factoría` para que devuelva un `ProductoConcreto`.



Prototype (Prototipo)

Este patrón tiene como finalidad crear nuevos objetos duplicándolos, clonando una instancia creada previamente.

Motivación

Este patrón resulta útil en escenarios donde es preciso abstraer la lógica que decide qué tipos de objetos utilizará una aplicación, de la lógica que luego usarán esos objetos en su ejecución. Los motivos de esta separación pueden ser variados, por ejemplo, puede ser que la aplicación deba basarse en alguna configuración o parámetro en tiempo de ejecución para decidir el tipo de objetos que se debe crear. En ese caso, la aplicación necesitará crear nuevos objetos a partir de modelos. Estos modelos, o prototipos, son clonados y el nuevo objeto será una copia exacta de los mismos, con el mismo estado. Esto resulta interesante para crear, en tiempo de ejecución, copias de objetos concretos inicialmente fijados, o

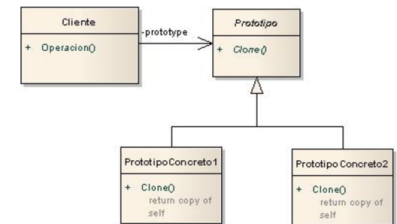
también cuando sólo existe un nº pequeño de combinaciones diferentes de estado para las instancias de una clase.

Aplicabilidad

- Si el sistema debe de ser independiente de cómo, dónde y cuándo se crean sus productos.
- Permite especificar instancias en tiempos de ejecución.
- Se quiere reducir el número de clases.
- Si las instancias que se generan tienen estados limitados.

Participantes

- **Prototipo**: declara una interfaz para clonarse.
- **PrototipoConcreto**: implementa la operación para clonarse.
- **Cliente**: crea un nuevo objeto pidiéndole a un Prototipo para que se clone.



Singleton (Instancia única)

El patrón instancia única está diseñado para restringir la creación de objetos pertenecientes a una clase o el valor de un tipo a un único objeto. Su intención consiste en garantizar que una clase sólo tenga una instancia y proporcionar un punto de acceso global a ella. No se encarga de la creación de objetos en sí, sino que se enfoca en la restricción en la creación de un objeto.

Motivación

En algunas ocasiones es muy importante poder garantizar que solo existe una instancia de una clase. Imaginamos la situación de varias impresoras disponibles cuando solo existe un pool que maneja la impresión. Es necesario asegurar que solo existe un objeto dentro del gestor de la impresión. Basándonos en el ejemplo, necesitamos un patrón que nos permita controlar las situaciones que exigen un control de acceso a una instancia bandera, muy habitual en sistemas concurrentes.

Aplicabilidad

Este patrón es aplicable en sistemas en los que se desea poder garantizar que solo existe una instancia de una clase.

Participantes

- **Singleton**: define una operación de clase getInstance() que permite a los clientes acceder a ella y además, es responsable de la creación de la instancia única.

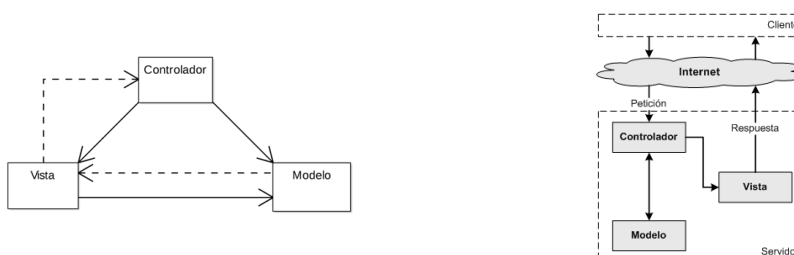


Model View Controller (MVC)

Este patrón, o modelo de abstracción, de desarrollo de software separa los datos de una aplicación, la interfaz de usuario y la lógica de negocio en tres componentes distintos.

Se ve frecuentemente en aplicaciones web, donde:

- El **Modelo** contiene una representación de los datos que maneja el sistema, su lógica de negocio, y sus mecanismos de persistencia.
- La **Vista**, o interfaz de usuario, compone la información que se envía al cliente y los mecanismos interacción con éste.
- El **Controlador**, actúa como intermediario entre el Modelo y la Vista, gestionando el flujo de información entre ellos y las transformaciones para adaptar los datos a las necesidades de cada uno.



2. Patrones estructurales



Adapter o Wrapper (Adaptador o Envoltorio)

Este patrón convierte la interfaz de una clase en otra interfaz para adaptarla a las necesidades de un desarrollo concreto. El patrón Adaptador permite que clases con interfaces incompatibles puedan trabajar juntas.

Motivación

Es muy frecuente la necesidad de adaptadores para elementos de la vida cotidiana: cargadores de baterías, tipos de enchufe, etc. Si traspasamos esta visión al mundo del software, en algunas ocasiones un conjunto de clases no es reutilizable simplemente por la interfaz que no concuerda con el dominio específico que una aplicación requiere. Es necesario crear un patrón que facilite esta reutilización y que permita no modificar la estructura de códigos del cliente y del servicio.

Aplicabilidad

- Si se quiere reutilizar una clase pero su interfaz no concuerda con nuestras necesidades.
- Cuando se desea crear una clase reutilizable que coopera con clases no relacionadas, es decir, clases que no tienen necesariamente interfaces compatibles.
- Cuando se quiera utilizar un componente de caja negra.

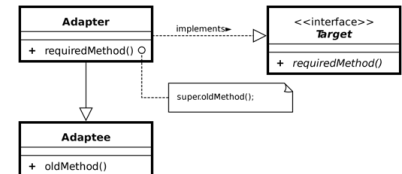
Estructura

Una **clase adaptadora** hereda de la clase o clases que desea adaptar, ofreciendo una interfaz diferente a la clase o clases padre.

Un **objeto adaptador** es un objeto que contiene uno o varios objetos de la clase o clases que se desean adaptar. La interfaz del objeto adaptador será el requerido, e internamente se realizarán las operaciones de conversión necesarias para adaptar las peticiones a la interfaz del objeto y objetos adaptados.

Participantes

- **Target** define la interfaz específica del dominio que Cliente usa.
- **Cliente** colabora con la conformación de objetos para la interfaz Target.
- **Adaptee** define una interfaz existente que necesita adaptarse.
- **Adapter** adapta la interfaz de Adaptee a la interfaz Target.



Bridge (Puente)

La idea de este patrón es desacoplar una abstracción de su implementación, de manera que ambas puedan ser modificadas independientemente sin necesidad de que la alteración de una afecte a la otra.

Motivación

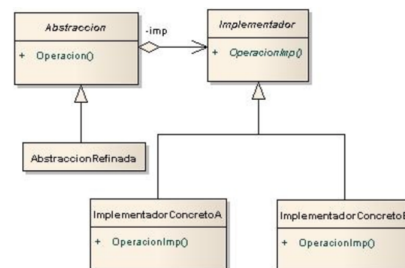
Cuando una abstracción puede tener varias posibles implementaciones, normalmente hacemos uso de la herencia para acomodar esta necesidad. Una clase abstracta define la interfaz de la abstracción y las subclases concretas lo implementan. Esto no es lo suficientemente flexible, es difícil de mantener y modificar y no permite reutilizar los componentes. Necesitamos un patrón que solucione esta problemática.

Aplicabilidad

- Se desea evitar un enlace permanente entre la abstracción y su implementación. Esto puede ser debido a que la implementación debe ser seleccionada o cambiada en tiempo de ejecución.
- Tanto las abstracciones como sus implementaciones deben ser extensibles por medio de subclases. En este caso, el patrón permite combinar abstracciones e implementaciones diferentes y extenderlas independientemente.
- Cambios en la implementación de una abstracción no deben impactar en los clientes, es decir, su código no debe tener que ser recompilado.
- Se desea esconder la implementación de una abstracción completamente a los clientes. En C++, la representación de una clase es visible en la interface de la clase.
- Se desea compartir una implementación entre múltiples objetos (quizá usando contadores) y este hecho debe ser escondido a los clientes.

Participantes

- **Abstracción:** define una interfaz abstracta. Mantiene una referencia a un objeto de tipo Implementador.
- **AbstracciónRefinada:** extiende la interfaz Abstracción.
- **Implementador:** define la interfaz para la implementación de clases. Esta interfaz no tiene que corresponder exactamente con la interfaz de Abstracción; de hecho, las dos interfaces pueden ser bastante diferentes. Típicamente, la interfaz Implementador provee sólo operaciones primitivas, y Abstracción define operaciones de alto nivel basadas en estas primitivas.
- **ImplementadorConcreto:** implementa la interfaz de Implementador y define su implementación concreta.



Composite (Objeto compuesto)

El patrón sirve para construir objetos complejos a partir de otros más simples y similares entre sí, gracias a la composición recursiva y a una estructura en forma de árbol. Esto simplifica el tratamiento de los objetos creados, ya que, al poseer todos ellos una interfaz común, se tratan todos de la misma manera.

Motivación

En una aplicación gráfica de componentes sencillos (líneas, texto...), imaginemos que necesitamos crear una serie de clases para guardar información acerca de unas figuras geométricas. Se crearán las clases *Círculo*, *Cuadrado* y *Triángulo*, que heredarán todas de la clase *Figura* y que implementarán el método *pintar()*. Además, se necesita poder tratar también grupos de imágenes, ya que nuestro programa permite seleccionar varias de estas figuras a la vez para moverlas por la pantalla. Para la implementación de estos grupos de Figuras podríamos caer en la tentación de crear una clase particular separada de las anteriores llamada *GrupoDeImágenes*, también con un método *pintar()*.

Podríamos pensar la idea de separar en clases privadas componentes (figuras) y contenedores (grupos) pero tiene el problema de que, para cada uno de los 2 tipos de objeto, el método *pintar()* tendrá una implementación diferente, aumentando la complejidad del sistema. Lo lógico es crear una clase abstracta que represente componentes y contenedores de la cual todas heredan, y que define sus operaciones. Por ejemplo, en este caso, se podría crear una clase *Gráfico* de la que hereden tanto la clase *Figura* como la clase *GrupoDeImágenes*, y que incluya el método *pintar()*. Además, ésta última tendría una relación todo-parte de multiplicidad con *Gráfico*: un *GrupoDeImágenes* contendría varios *Gráficos*, ya fuesen estos *Cuadrados*, *Triángulos*, u otras clases *GrupoDeImágenes*.

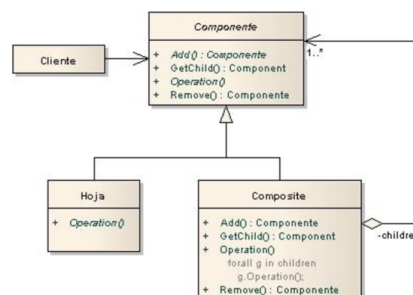
De esta forma, el patrón Composite da una solución elegante a este problema, de la que además resulta en una implementación más sencilla.

Aplicabilidad

- Se quiere realizar jerarquías de objetos del tipo "todo-parte".
- Se quiere ser capaz de ignorar la diferencia entre los objetos individuales y composiciones de objetos. Los clientes tratarán a todos los objetos de la estructura compuesta uniformemente.

Participantes

- **Componente**: define la interfaz común para los objetos de la composición. Además, define la interfaz para acceder y gestionar los hijos. Implementa un comportamiento por defecto común a las subclases.
- **Hoja**: representa los objetos sin hijos de la composición. Define el comportamiento de los mismos.
- **Composite**: define el comportamiento para los componentes que tienen hijos. Almacena los componentes con hijos e implementa las operaciones para la gestión de hijos.
- **Cliente**: manipula los objetos de la composición a través de la interfaz.



Decorator (Decorador)

El patrón responde a la necesidad de añadir dinámicamente funcionalidad (responsabilidades) a un objeto. Esto nos permite no tener que crear sucesivas clases que hereden de la primera incorporando la nueva funcionalidad, sino otras que la implementan y se asocian a la primera.

Motivación

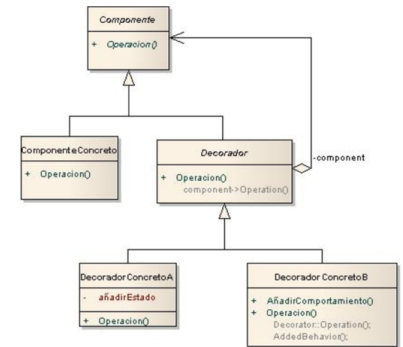
A veces se quiere añadir funcionalidad a un objeto concreto, no a una clase entera. Supongamos, por ejemplo, un kit de herramientas de una interfaz gráfica para GUIs que proporciona soporte para marcos, barras de desplazamiento, etc. Podríamos intentar resolver esta situación mediante el empleo de la herencia entre clases, pero no es posible ya que este mecanismo no es flexible y la funcionalidad se añade estáticamente. Si definimos una clase que envuelva al componente y proporcione la funcionalidad deseada, tendremos un diseño más flexible y transparente. El patrón Decorador nos proporciona esta solución.

Aplicabilidad

- Añadir responsabilidades a objetos concretos de manera dinámica y transparente, esto es, sin afectar a otros objetos.
- Para el tratamiento de responsabilidades que se pueden otorgar o derogar.
- Cuando la herencia sea muy compleja porque implique la creación de múltiples subclases para dar completitud a todas las combinaciones posibles.

Participantes

- **Componente:** define la interfaz de los objetos a los que se puede añadir responsabilidades de manera dinámica.
- **ComponenteConcreto:** define un objeto al que añadir responsabilidades de manera dinámica.
- **Decorator:** mantiene una referencia al objeto componente y define una interfaz conforme a la del componente.
- **DecoratorConcreto:** añade responsabilidades al componente al que referencia.



Facade (Fachada)

Este patrón sirve para proveer de una interfaz unificada sencilla que haga de intermediaria entre un cliente y una interfaz o grupo de interfaces más complejas.

Motivación

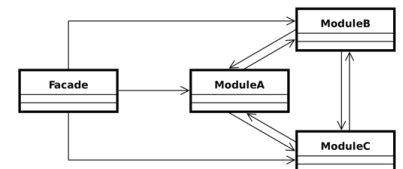
Estructurar un sistema complejo en pequeños subsistemas reduce la complejidad global. Un objetivo común del diseño es reducir y minimizar las comunicaciones entre subsistemas. Una forma de conseguirlo puede ser introduciendo un objeto Fachada que provea una interfaz simple ampliando la facilidad del sistema.

Aplicabilidad

- Se quiere proporcionar una interfaz sencilla para un subsistema complejo.
- Se quiere desacoplar un subsistema de sus clientes o de otros subsistemas convirtiéndolo en más independiente y portable.
- Se quiera dividir el sistema por niveles, actuando las fachadas como entradas a cada nivel.

Participantes

- **Fachada:** delegan la petición del cliente en objetos de los subsistemas.
- **Módulos:** implementan las funcionalidades.



Flyweight (Peso ligero)

Este patrón sirve para eliminar o reducir la redundancia cuando tenemos gran cantidad de objetos que contienen información idéntica, además de lograr un equilibrio entre flexibilidad y rendimiento (uso de recursos).

Motivación

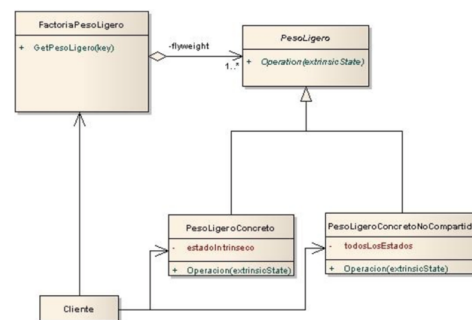
Describe como almacenar un gran nº de objetos sin un gran coste. Para conseguir esto se utilizan objetos que almacenan los estados compartidos y que pueden ser utilizados por varios objetos simultáneamente.

Aplicabilidad

- Se utiliza un gran número de objetos.
- El coste del almacenamiento es alto debido a la cantidad de objetos.
- La mayoría de los estados de los objetos pueden ser creados como comunes.
- Muchos objetos pueden ser reemplazados por unos pocos una vez que han sido borrados los estados comunes.
- La mayor parte del estado del objeto puede ser extrínseco.

Participantes

- **PesoLigeroConcreto**: cualquier estado que almacene debe de ser independiente de contexto. Deben de ser compartibles. Implemente la interfaz de PesoLigero.
- **PesoLigero**: declaran una interfaz a través de la cual los PesosLigeros pueden recibir y actuar sobre estados no compartidos.
- **PesoLigeroConcretoNoCompartido**: no todas las subclases de PesoLigero tiene que ser compartidas.
- **Cliente**: contiene las referencias a los PesosLigeros.
- **FactoriaPesoLigero**: crea y gestiona los objetos PesoLigero, garantiza que los objetos PesoLigero se comparten de forma adecuada.



Proxy

Este patrón nos proporciona un sustituto o representante de otro objeto para controlar el acceso a este.

Motivación

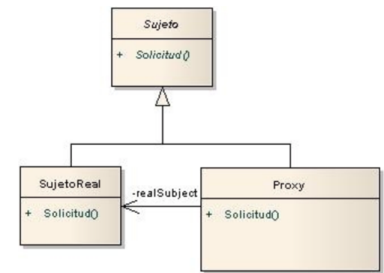
El principal motivo de no permitir el acceso directo a un objeto es reducir el coste que supone la creación y mantenimiento del mismo hasta que realmente es necesario. Imaginemos un documento con muchas imágenes embebidas, supone un consumo de recursos, pero a su vez es necesaria la rapidez a la hora de abrir el documento. Si partimos de la idea de que no todas las imágenes se ven en el documento a la vez, solo se irán cargando aquellas que sean necesarias. Esta idea se puede implementar usando este patrón.

Aplicabilidad

- Útil cuando se desea retrasar la instanciación de un objeto hasta que sea necesario usarlo.
- Proporciona un representante local de un objeto situado en otro espacio de direcciones.
- Uso en sistemas concurrentes mediante cerrojo, controlando el acceso al objeto original.

Participantes

- **Sujeto:** define la interfaz común para Proxy y SujetoReal, de manera que pueda usarse un Proxy donde se espera un SujetoReal.
- **SujetoReal:** define el objeto real representado.
- **Proxy:** mantiene una referencia que permite al proxy acceder al objeto real. Proporciona una interfaz idéntica a la del Sujeto de manera que pueda ser sustituido por un SujetoReal. Controla el acceso al SujetoReal, puede ser el responsable de su creación y borrado.



3. Patrones de comportamiento



Chain of Responsibility (Cadena de responsabilidades)

El patrón de diseño Cadena de responsabilidades permite establecer una cadena de objetos receptores a través de los cuales se pasa una petición formulada por un objeto emisor. Cualquiera de los objetos receptores puede responder a la petición en función de un criterio establecido.

Motivación

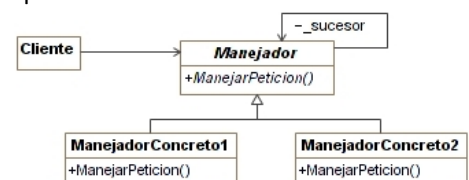
Imaginemos un contexto gráfico donde se puede obtener información de ayuda de cada elemento clicando sobre él. La información dependerá de la parte de la interfaz donde se pinche. Puede darse la situación que 2 "botones" iguales difieran en la información a mostrar. Para este tipo de problemas necesitamos un patrón que permita manejar dónde se produce un evento, quién es el responsable del mismo y cuál es la respuesta adecuada al mismo.

Aplicabilidad

- Más de un objeto necesita manejar una respuesta y el manejador no es conocido a priori.
- Se quiere poder realizar una petición sin conocer a quién hay que solicitarlo.
- El conjunto de objetos que pueden procesar una respuesta pueden ser especificados de forma dinámica.

Participantes

- **Cliente:** inicia la petición que llega a la cadena en busca del responsable.
- **Manejador:** define una interfaz para manejar peticiones.
- **ManejadorConcreto:** define las responsabilidades de cada componente. Si puede manejar una petición, la procesa, en caso contrario busca al siguiente.





Command (Comando)

Este patrón permite solicitar una operación a un objeto sin conocer realmente el contenido de esta operación, ni el receptor real de la misma. Para ello se encapsula la petición como un objeto, con lo que además se facilita la parametrización de los métodos.

Motivación

El concepto de comando es muy ambiguo y complejo pero está muy extendido. Podemos mencionar como ejemplos: intérpretes de comandos del sistema operativo, lenguajes de macros, SGBD y protocolos de servidores de internet.

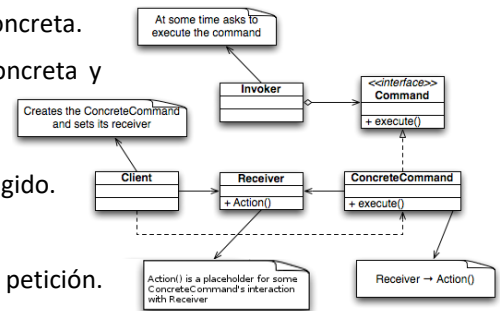
Es necesario encontrar un patrón que permita parametrizar a los clientes con distintas peticiones, hacer colas de peticiones, llevar un registro de las peticiones realizadas y que pueda deshacer el estado y el efecto de las mismas. El patrón Comando permite realizar todo esto.

Aplicabilidad

- Parametrizar objetos mediante una acción.
- Especificar, encolar y ejecutar peticiones en distintos momentos (el objeto Comando tiene un tiempo de vida distinto al de la petición).
- Independizar el momento de petición del de ejecución.
- Mantener operaciones que permitan deshacer la petición.
- Acciones que permitan la recuperación del sistema.
- Interfaz común que permita invocar las acciones de modo uniforme y extender el sistema de modo sencillo.

Participantes

- **Comando**: clase que ofrece una interfaz para la ejecución de órdenes. Define los métodos *do* y *undo* que se implementarán en cada clase concreta.
- **ComandoConcreto**: clase que implementa una orden concreta y sus métodos *do* y *undo*. Su constructor debe inicializar los parámetros de la orden.
- **Cliente**: crea un comando concreto e indica a quién va dirigido.
- **Invocador**: contiene el comando asociado a la petición.
- **Receptor**: sabe realizar las operaciones asociadas a una petición. Cualquier clase puede actuar como receptor.



Interpreter (Intérprete)

El Intérprete es un patrón de diseño que, dado un lenguaje, define una representación para su gramática junto con un intérprete del lenguaje. Se usa para definir un lenguaje para construir expresiones regulares que representen cadenas a buscar dentro de otras cadenas.

Además, en general, para definir un lenguaje que permita representar las distintas instancias de una familia de problemas.

Motivación

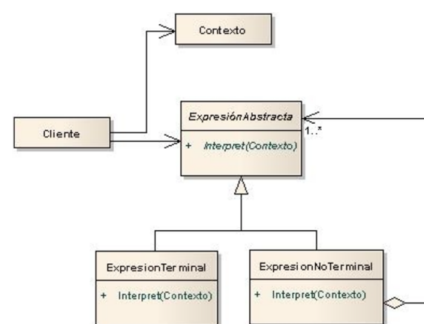
Existen problemas particulares que pueden expresarse en función de algún lenguaje. Es necesario construir un intérprete de dicho lenguaje. El patrón define reglas gramaticales del lenguaje, así como la realización y comprensión de sentencias del mismo.

Aplicabilidad

- Cuando tratamos con gramáticas simples, en caso contrario la mejor opción es utilizar parsers.
- La eficiencia no es uno de los aspectos más importantes. Hay que traducir el input a una forma inmediata.

Participantes

- **ExpresiónAbstracta**: declara una operación abstracta Intérprete que es común a todos los nodos del árbol de sintaxis abstracta.
- **ExpresiónTerminal**: una instancia es requerida por cada aparición en una sentencia. implementa una operación Intérprete asociada a cada símbolo terminal.
- **ExpresiónNoTerminal**: para cada regla es necesario un tipo de clase.
- **Cliente**: construye el árbol sintáctico de ExpresionesNoTerminales, e instancias de la clase ExpresiónTerminal.
- **Contexto**: contiene información global para el interpretador.



Iterator (Iterador)

Es un patrón define una interfaz que declara los métodos necesarios para acceder secuencialmente a un grupo de objetos de una colección.

Motivación

Debemos de disponer de un medio para navegar por los datos de una lista sin exponer su estructura interna. Así mismo, se debe recorrer la lista de varias maneras pero sin que esto signifique añadir operaciones a la lista por cada tipo de recorrido. En algunos casos nos interesa mantener varios recorridos simultáneamente.

El patrón Iterador permite hacer todo esto. La solución que propone es dar la responsabilidad de recorrer la lista a un objeto Iterador. Éste define una interfaz para acceder a los elementos de la lista.

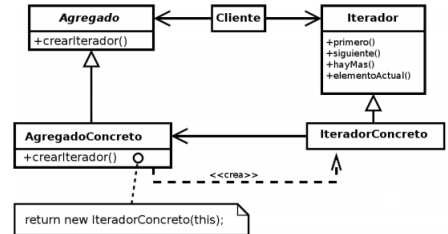
Aplicabilidad

- Para acceder a la información de un agregado sin exponer su estructura interna.

- Se quiere permitir varios tipos de recorrido sobre un agregado.
- Proporcionar una interfaz uniforme para recorrer diferentes tipos de agregados.

Participantes

- **Agregado:** define la interfaz para crear un objeto Iterador.
- **Iterador:** define la interfaz para agregar y recorrer elementos.
- **AgregadoConcreto:** implementa la interfaz para crear un objeto Iterador.
- **IteradorConcreto:** implementa la interfaz de Iterador y mantiene la posición actual del recorrido.



Mediator (Mediador)

Un Mediador es un patrón de diseño que coordina las relaciones entre sus asociados. Define un objeto que encapsula cómo interactúan un conjunto de objetos. Promueve un bajo acoplamiento al evitar que los objetos se refieran unos a otros explícitamente, y permite variar la interacción entre ellos de forma independiente.

Motivación

Cuando muchos objetos interactúan con otros objetos, se puede formar una estructura muy compleja, con objetos con muchas conexiones con otros objetos. En un caso extremo cada objeto puede conocer a todos los demás objetos. Para evitar esto el patrón Mediator encapsula el comportamiento de todo un conjunto de objetos en un solo objeto.

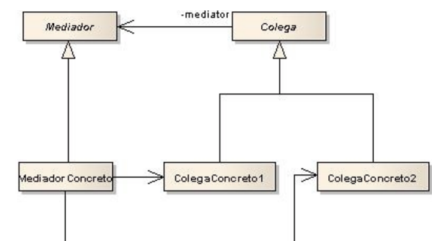
Los objetos envían y reciben peticiones a través del Mediador, este implementa el comportamiento cooperativo encaminando las peticiones a los objetos deseados.

Aplicabilidad

- Un conjunto grande de objetos se comunica de una forma bien definida, pero compleja.
- Existe dificultad para reutilizar objetos, ya que nos referimos a varios objetos para comunicarnos.
- El comportamiento de muchos objetos (que está distribuido en varias clases) puede resumirse en una o varias por subclasificación.

Participantes

- **Mediador:** define una interfaz para comunicarse con los otros objetos.
- **Colega:** cada Colega conoce su Mediador, y usa a este para comunicarse con otros Colegas.
- **ColegaConcreto:** implementa el comportamiento del Colega.
- **MediadorConcreto:** implementa el comportamiento cooperativo entre los Colegas (como se comunican entre ellos). Además los conoce y mantiene.





Memento (Recuerdo)

El patrón Memento guarda parte o todo el estado interno de un objeto, para que este estado pueda ser restaurado posteriormente. Esta operación debe ocurrir sin romper el principio del encapsulamiento.

Motivación

Muchas veces es necesario guardar el estado interno de un objeto. Esto es debido a que, tiempo después, se necesita restaurar el estado del objeto al que previamente se ha guardado. Por ejemplo, una aplicación de composición de figuras geométricas, donde el usuario hace sucesivas modificaciones a una composición, graficando nuevas líneas, círculos y rectángulos. Después de cierto tiempo, el usuario logra una composición “casi perfecta”, pero decide alcanzar la perfección, así que pinta una línea y esta no le sale como él esperaba. El usuario querría regresar al instante en que su “creación” era una obra de arte. Para dar solución a este problema, antes de que el usuario agregue una nueva figura geométrica a la composición se debería guardar el estado de la composición y entonces siempre se tendría la posibilidad de regresar hacia atrás y restaurar la composición a su estado anterior.

Para lograr esto, sería necesario guardar la lista de figuras geométricas y el orden en que se encuentran en la composición, con información específica de cada una de ellas. En el caso de un círculo tendríamos que guardar la posición (x,y), el radio, color y relleno, para un rectángulo la posición (x,y), el ancho, el largo, color y relleno. Para lograr esto tenemos 3 alternativas: (1ª) consiste en obtener la lista de figuras de la composición y obtener su estado, esto sería muy complejo y además va en contra del principio de encapsulamiento; (2ª) es que la composición se encargue de ir guardando su estado interno cada vez, esta no es una buena alternativa ya que la clase sería muy compleja y estaría asumiendo responsabilidades que no le corresponden; (3ª) es la mejor: la composición crea un objeto Memento y almacena su estado interno en él, la aplicación mantiene una lista de los objetos Memento, de tal manera que cuando el usuario realice una operación de “deshacer”, la aplicación restaura el estado de la composición con lo almacenado por el objeto Memento.

Aplicabilidad

- Todo o parte del objeto debe de ser almacenado para una posible restauración del mismo.
- Cuando una interfaz directa para obtener el estado de un objeto exponga detalles de su implementación.

Participantes

- **Memento:** almacena el estado de un objeto Originador. Memento almacena todo o parte de Originador. Tiene 2 interfaces, una para Aplicación que le permite comunicarse con otros objetos y otra para Originador que le permite almacenar el estado.
- **Originador:** crea un objeto Memento con una copia de su estado. Usa Memento para restaurar el estado almacenado.
- **Aplicación:** mantiene a Memento pero no opera con su contenido.





Observer (Observador)

Este patrón define una dependencia del tipo *uno-a-muchos* entre objetos, de manera que cuando uno de los objetos cambia su estado, el *Observador* se encarga de notificar este cambio al resto de los objetos dependientes. El objetivo de este patrón es desacoplar la clase de los objetos clientes del objeto, aumentando la modularidad del lenguaje, así como evitar bucles de actualización.

Motivación

Es necesario mantener la consistencia entre objetos relacionados sin aumentar el acoplamiento de las clases. Si imaginamos la relación entre la capa de presentación de un IU y los datos subyacentes en una representación gráfica observamos rápidamente qué pretende solucionar este patrón.

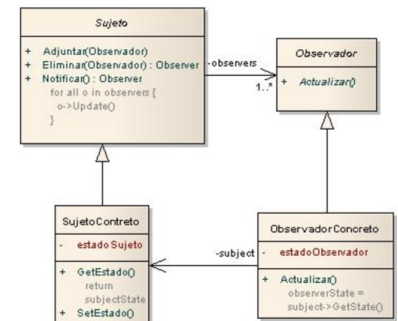
La interpretación en diferentes diagramas de un conjunto de datos depende de los datos de los objetos. Cualquier modificación en los mismos produce un efecto en la representación. El patrón Observador establece las relaciones entre los cambios del sujeto y su efecto posterior.

Aplicabilidad

- Cuando una abstracción tiene 2 aspectos dependientes el uno del otro. Encapsular los aspectos en objetos distintos permite cambiarlos y reutilizarlos.
- Cuando cambiar un objeto implica cambiar otros pero se desconoce el nº exacto a cambiar.
- Cuando un objeto debe ser capaz de notificar algo a otros sin hacer suposiciones sobre quienes son dichos objetos, es decir, cuando se quiere bajo acoplamiento.

Participantes

- **Observador:** define la interfaz de los objetos a los que hay que notificar cambios del Sujeto.
- **ObservadorConcreto:** mantiene una relación con un SujetoConcreto. Mantiene el estado del SujetoConcreto que le es de interés. Implementa Observador para mantener su estado coherente con el del Sujeto.
- **Sujeto:** conoce a sus observadores. Tiene una interfaz para ir añadiendo y eliminando observadores.
- **SujetoConcreto:** envía notificaciones a sus observadores cuando su estado cambia.



State (Estado)

Este patrón se utiliza cuando el comportamiento de un objeto cambia dependiendo del estado del mismo.

Motivación

Imaginemos la situación de algún objeto con varios estados en función de mensajes externos. Por ejemplo, una comunicación que usa el protocolo TCPConnection, que representa una conexión de red. Un objeto de esta clase tendrá diferentes respuestas según su estado (Listening, Closed o Established). Una llamada al método open de un objeto de la clase TCPConnection diferirá en su comportamiento si la conexión se encuentra en el estado Close o en el estado Established.

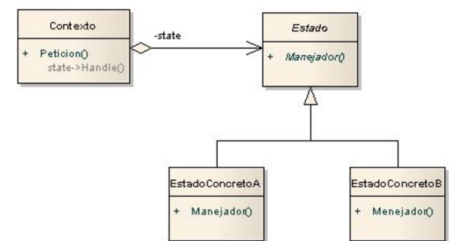
La idea sería introducir una clase que, en función del estado y de los condicionantes que presenta el mismo, haga que el sistema responda adecuadamente. El patrón Estado proporciona una solución a esta problemática.

Aplicabilidad

- El comportamiento de un objeto depende del estado en el que se encuentra y este estado se modifica en función de las transiciones de estado.
- Si existen muchas operaciones con la misma lógica, el patrón permite introducir esta lógica en una clase separada.

Participantes

- **Contexto**: define la interfaz de interés para el cliente. Mantiene una instancia EstadoConcreto con el estado actual.
- **Estado**: define una interfaz para el encapsulamiento del comportamiento asociado a un estado particular del contexto.
- **EstadoConcreto**: implementa el comportamiento de un estado de contexto.



Strategy (Estrategia)

Es un patrón que permite mantener un conjunto de algoritmos de los que el objeto cliente puede elegir aquel que le conviene e intercambiarlo según sus necesidades. Los distintos algoritmos se encapsulan y el cliente trabaja contra un objeto Contexto. El cliente puede elegir el algoritmo que prefiera de entre los disponibles o puede ser el mismo objeto Contexto el que elija el más apropiado para cada situación.

Cualquier programa que ofrezca un servicio o función determinada, que pueda ser realizada de varias maneras, es candidato a utilizar el patrón Estrategia. Puede haber cualquier nº de estrategias y cualquiera de ellas podrá ser intercambiada por otra en cualquier momento, incluso en tiempo de ejecución.

Motivación

En muchas ocasiones una solución es válida solo para unas determinadas circunstancias. Si imaginamos el caso de un flujo de texto que queremos dividir en líneas podríamos solucionarlo codificando los algoritmos necesarios para cada posible situación en las clases afectadas. Pero esto lleva asociado las siguientes consecuencias: los clientes se hacen más

complejos, distintos algoritmos serán apropiados en distintos momentos, hay un mantenimiento muy grande y es difícil añadir nuevos algoritmos y modificar los existentes.

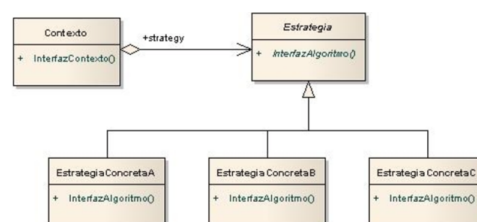
La mejor solución es definir clases que encapsulen los algoritmos y que sean llamadas por las clases que los necesitan, obteniendo así un desacoplamiento ante la aparición de nuevos algoritmos.

Aplicabilidad

- Varias clases relacionadas que solo difieren en su comportamiento. Estrategia permite configurar a una clase con uno de los comportamientos.
- Se necesitan variantes de un mismo algoritmo. Se implementan como una jerarquía de clases.
- Un algoritmo usa datos que un cliente no debe conocer.
- Una clase define muchos comportamientos basados en condicionales. Mover estas condiciones a la clase de Estrategia.

Participantes

- **Contexto:** está configurado con un objeto de la EstrategiaConcreta. Mantienen una relación con el objeto Estrategia y define una interfaz para que Estrategia acceda a sus datos.
- **Estrategia:** define una interfaz común a los algoritmos que soporta.
- **EstrategiaConcreta:** define una implementación a un algoritmo mediante la interfaz de Estrategia.



Template Method (Método plantilla)

Es un patrón de diseño que define una estructura algorítmica en una superclase, delegando la implementación a las subclases, es decir, define una serie de pasos que serán redefinidos en las subclases.

Motivación

Imaginemos una situación donde usamos una aplicación que accede a documentación. La clase que mantiene la aplicación será la responsable de abrir los documentos, mientras que una clase Documentación será la encargada de mantener la información de los mismos. Cada aplicación tendrá su clase de Documentación asociada. En definitiva, vamos construyendo métodos que incluyen en su código llamadas a métodos abstractos. Estos métodos son los denominados Plantilla.

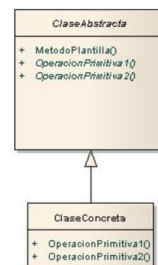
Aplicabilidad

- Factorizar el comportamiento común de varias subclases.
- Implementar las partes fijas de un algoritmo una sola vez y dejar que las subclases implementen las partes variables.

- Cuando se diseña una biblioteca. Algunas de las clases pueden tener un comportamiento que dependa de la aplicación que la use. En tal caso, este patrón obliga a que el programador proporcione dicho comportamiento.
- Definir una estructura lo más reutilizable posible para la autenticación de usuarios.
- Controlar las ampliaciones de las subclases, convirtiendo en métodos plantillas aquellos métodos que pueden ser redefinidos.

Participantes

- **ClaseAbstracta:** implementa un método plantilla que define el esqueleto de un algoritmo y define los métodos abstractos que implementa las ClasesConcretas.
- **ClaseConcreta:** implementa los métodos abstractos para realizar los pasos del algoritmo que son específicos de la subclase.



Visitor (Visitante)

Es un patrón que permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.

Motivación

Consideremos una aplicación relacionada con compiladores. Estos representan los programas de árboles de sintaxis abstracta sobre los que ejecutan operaciones para realizar su funcionalidad. Muchas de estas operaciones tienen un contexto propio y por lo tanto necesitan diferenciar cierta información (tipo de nodo del árbol). Esto genera un conflicto, ya que lo convierte en difícil de implementar y mantener. La aparición de nuevos elementos supondría recompilar todas las clases existentes.

El patrón Visitor propone una solución a este problema. Pretende independizar las clases de las operaciones que se ejecutan sobre ellas.

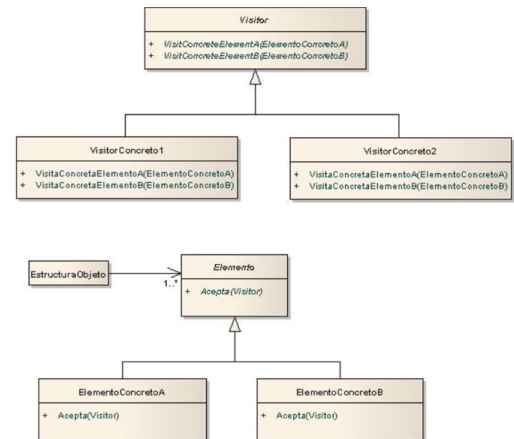
Aplicabilidad

- Una estructura de objetos contiene muchas clases de objetos con interfaces distintas y se quiere realizar sobre ellos operaciones que son distintas en cada clase concreta.
- Se quieren realizar muchas operaciones distintas sobre los objetos de una estructura, sin incluir dichas operaciones en las clases.
- Las clases que forman la estructura de objetos no cambian pero las operaciones sobre ellas sí.

Participantes

- **Visitor:** define una operación de visita para cada clase de elemento concreto en la estructura de objetos.
- **VisitorConcreto:** implementa la interfaz de Visitor. Cada operación implementa un fragmento de la labor global del VisitorConcreto pudiendo almacenar información local.

- **Elemento:** define la operación acepta con el Visitor de argumento.
- **ElementoConcreto:** implementa la operación acepta.
- **EstructuraObjeto:** gestiona la estructura de objetos y puede enumerar sus elementos. Puede ser un compuesto (Composite) o una colección de objetos. Puede ofrecer una interfaz que permita al Visitor visitar a sus elementos.



4. Esquema patrones de diseño

CREACIONALES

- **Abstract Factory:** define una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas.
- **Builder:** permite la creación de objetos complejos paso a paso.
- **Factory Method:** crea objetos de un subtipo, pero deja que las subclasses decidan qué clase instanciar.
- **Prototype:** permite crear nuevos objetos clonando una instancia existente.
- **Singleton:** asegura que una clase tenga solo una instancia única.
- **Model View Controller:** separa los datos, la IU y la lógica de negocio en 3 componentes distintos.

ESTRUCTURALES

- **Adapter o Wrapper:** convierte la interfaz de una clase en otra interfaz que el cliente espera.
- **Bridge:** desacopla una abstracción de su implementación para que ambas puedan variar independientemente.
- **Composite:** compone objetos en estructuras de árbol para representar jerarquías parte-todo.
- **Decorator:** añade responsabilidades adicionales a un objeto de manera dinámica.
- **Facade:** proporciona una interfaz unificada para un conjunto de interfaces en un subsistema.
- **Flyweight:** elimina redundancias cuando existen muchos objetos con información idéntica.
- **Proxy:** Proporciona un representante de otro objeto para controlar el acceso a este.

COMPORTAMIENTO

- **Chain of Responsibility:** permite establecer una cadena de objetos receptores para que puedan tratar una petición enviada por un emisor.
- **Command:** permite solicitar una operación a un objeto sin conocer realmente el contenido de esta operación, ni el receptor real de la misma.
- **Interpreter:** para un lenguaje define una representación para su gramática y un intérprete.
- **Iterator:** proporciona una manera de acceder secuencialmente a una colección de objetos.
- **Mediator:** define un objeto que encapsula cómo interactúan un conjunto de objetos.
- **Memento:** guarda el estado interno de un objeto para poder ser restaurado posteriormente.
- **Observer:** define una dependencia 1 a N entre objetos para que cuando un objeto cambie de estado, todos sus dependientes sean notificados.
- **State:** permite a un objeto alterar su comportamiento cuando su estado interno cambia.
- **Strategy:** define una familia de algoritmos y los hace intercambiables.
- **Template Method:** define una estructura en la superclase y delega la implementación a las subclasses.
- **Visitor:** permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.

6. LENGUAJE DE MODELADO UNIFICADO (UML)

Un **modelo** es una simplificación de la realidad. El modelo nos proporciona los planos de un sistema, desde los más generales, que proporcionan una visión general del sistema, hasta los más detallados.

En un modelo se han de incluir los elementos que tengan más relevancia y omitir los que no son interesantes para el nivel de abstracción que se ha elegido.

UML (Unified Modeling Language) es un lenguaje de propósito general que ayuda a especificar, visualizar y documentar modelos de sistemas software, incluido su estructura y diseño, de tal forma que se unifiquen todos sus requerimientos.

Por un lado, el lenguaje de modelado puede servir de modelo para un proyecto y garantizar así una arquitectura de información estructurada; por el otro, ayuda a los desarrolladores a presentar la descripción del sistema de una manera que sea comprensible para quienes están fuera del campo. UML se utiliza principalmente en el desarrollo de software orientado a objetos. Al ampliar el estándar en la versión 2.0, también es adecuado para visualizar procesos empresariales.

Desde finales de la década de 1980 hasta la de 1990 se desarrollaron muchas técnicas de modelado OO. El resultado fue una confusa abundancia de métodos que apenas eran comparables entre sí. Destacamos los siguientes métodos:

- **Booch**, de Grady Booch.
- **OMT** (Object Modeling Technique), de James Rumbaugh.
- **OOSE** (Object-Oriented Software Engineering), de Ivar Jacobson.

UML comenzó a gestarse a mediados de los noventa, cuando James Rumbaugh, Grady Booch e Ivar Jacobson se unieron en Rational. Esto derivó en el primer borrador de UML apareció en octubre de 1995.

Este lenguaje se abrió a la colaboración de otras empresas para que aportaran sus ideas. Todas estas colaboraciones condujeron a la definición de la primera versión de UML. En 1997 la versión UML 1.1 fue aprobada por la OMG convirtiéndose en la notación estándar de facto para el análisis y el diseño orientado a objetos.

OMG (Object Management Group - Grupo de Gestión de Objetos) es un consorcio dedicado al cuidado y el establecimiento de diversos estándares de tecnologías orientadas a objetos, tales como UML, XML, CORBA. Es una organización sin ánimo de lucro que promueve el uso de tecnología orientada a objetos mediante guías y especificaciones para las mismas. El grupo está formado por compañías y organizaciones de software como HP, Oracle o Microsoft.

Tras esto, los desarrolladores crearon un grupo de trabajo para mejorar el lenguaje a través de múltiples versiones. Las críticas existentes incluían una semántica imprecisa e

innecesariamente compleja, la falta de adaptabilidad y una estandarización inadecuada. Por lo tanto, se llevó a cabo una revisión a fondo. El resultado fue finalmente **UML 2.0**, que definió el **nuevo estándar** en 2005. La versión 2.4.1 constituye la base de la normalización:

- ISO/IEC **19505-1:2012 Infraestructura**.
- ISO/IEC **19505-2:2012 Superestructura**.

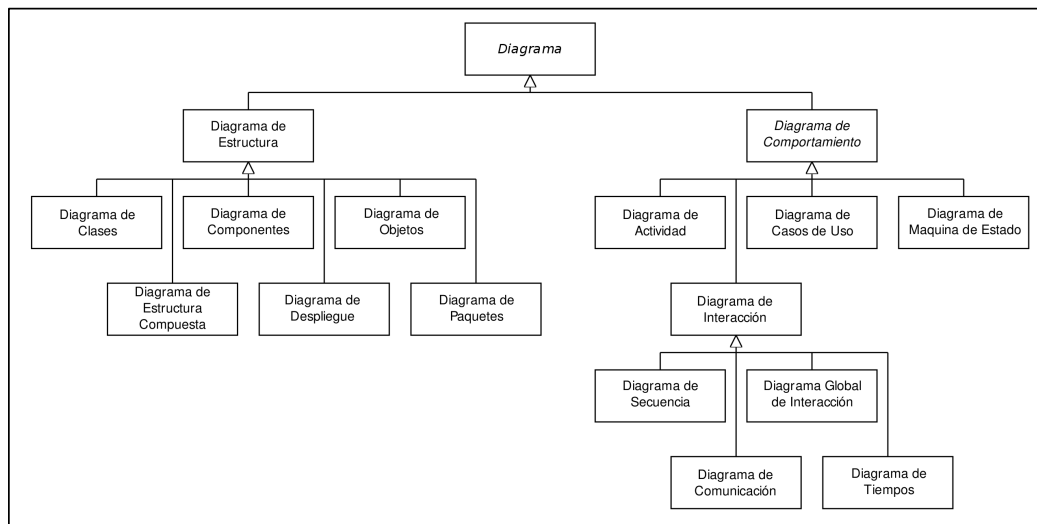
La versión actual de UML es la **2.5.1**, que apareció en diciembre de 2017.

Características de UML:

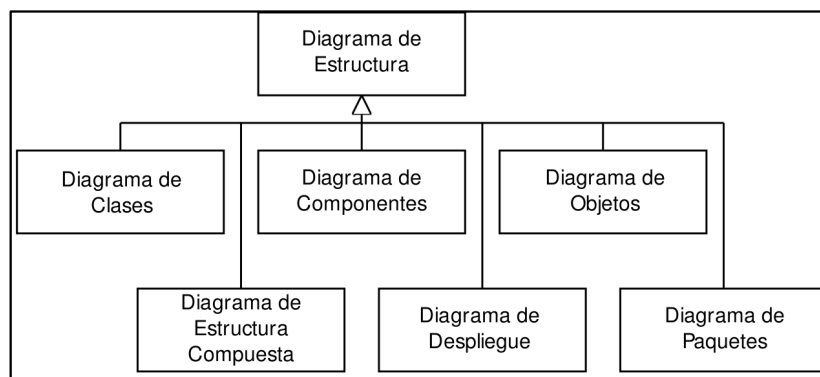
- Es un **lenguaje estándar** que sirve para escribir los planos del software.
- Es un **lenguaje gráfico** para visualizar, especificar, construir y documentar un sistema.
- Es **extensible**. Es posible personalizar el lenguaje para un dominio concreto.
- Modela sistemas, desde los requisitos hasta los artefactos ejecutables, utilizando **técnicas de Orientación a Objetos**.
- Es **escalable**: puede manejar sistemas grandes y pequeños.
- Es **exhaustivo**: describe todos los aspectos importantes del sistema.
- **Construido sobre la experiencia**: es la culminación de las mejores prácticas en la comunidad de OO de los últimos años.
- Permite ver el sistema desde distintas perspectivas (**vistas**). UML abarca todas las fases del ciclo de vida de un proyecto, soporta diferentes maneras de visualización dependiendo de quién tenga que interpretar los planos y en que fase del proyecto se encuentre.
- **No es una metodología**. Se puede usar en distintos tipos de metodología (cascada, iterativo...).

UML categoriza los diagramas en 2 tipos:

- **ESTRUCTURALES**: muestran la estructura estática del sistema a modelar.
- **De COMPORTAMIENTO**: muestran el comportamiento dinámico del sistema.



1. Diagramas de estructura (vista estática)



1 Diagrama de clases

El objetivo principal de este modelo es la representación de los aspectos estáticos del sistema, utilizando diversos mecanismos de abstracción (clasificación, generalización, agregación).

El diagrama de clases recoge las clases de objetos y sus asociaciones. En este diagrama se representa la estructura y el comportamiento de cada uno de los objetos del sistema y sus relaciones con los demás objetos, pero no muestra información temporal.

Con el fin de facilitar la comprensión del diagrama, se pueden incluir paquetes como elementos del mismo, donde cada uno de ellos agrupa un conjunto de clases. Este diagrama no refleja los comportamientos temporales de las clases, aunque para mostrarlos se puede utilizar un diagrama de máquina de estado.

Los **elementos** básicos del diagrama son:

- **Clases:** una clase describe un conjunto de objetos con propiedades (atributos) similares y un comportamiento común. Los objetos son instancias de las clases.

No existe un procedimiento inmediato que permita localizar las clases del diagrama de clases. Éstas suelen corresponderse con sustantivos que hacen referencia al ámbito del sistema de información y que se encuentran en los documentos de las especificaciones de requisitos y los casos de uso.

Dentro de la estructura de una clase se definen los atributos y las operaciones o métodos:

- o Los atributos de una clase representan los datos asociados a los objetos instanciados por esa clase.
- o Las operaciones o métodos representan las funciones o procesos propios de los objetos de una clase, caracterizando a dichos objetos.

El diagrama de clases permite representar clases abstractas. Una clase abstracta es una clase que no puede existir en la realidad, pero que es útil conceptualmente para el diseño del modelo orientado a objetos. Las clases abstractas no son instanciables directamente sino en sus descendientes. Una clase abstracta suele ser situada en la jerarquía de clases en una posición que le permita ser un depósito de métodos y atributos para ser compartidos o heredados por las subclases de nivel inferior.

Las clases y en general todos los elementos de los diagramas, pueden estar clasificados de acuerdo a varios criterios, como por ejemplo su objetivo dentro de un programa. Esta clasificación adicional se expresa mediante un estereotipo. Algunos de los autores de métodos OO, establecen una clasificación de todos los objetos que pueden aparecer en un modelo. Los tipos son:

- o Objetos Entidad. 
- o Objetos límite o interfaz. 
- o Objetos de control. 

Éstos son estereotipos de clases. Un estereotipo representa una la meta-clasificación de un elemento. Dependiendo de la herramienta utilizada, también se puede añadir información adicional a las clases para mostrar otras propiedades de las mismas, como son las reglas de negocio, responsabilidades, manejo de eventos, excepciones, etc.

- **Relaciones:** los tipos más importantes de relaciones estáticas entre clases son los siguientes:
 - o Asociación: las relaciones de asociación representan un conjunto de enlaces entre objetos o instancias de clases. Es el tipo de relación más general, y denota básicamente una dependencia semántica. Por ejemplo, una Persona trabaja para una Empresa.

Cada asociación puede presentar elementos adicionales que doten de mayor detalle al tipo de relación:

- **Rol** o nombre de la asociación: describe la semántica de la relación en el sentido indicado. Por ejemplo, la asociación entre Persona y Empresa recibe el nombre de trabaja para, como rol en ese sentido.
 - **Multiplicidad**: describe la cardinalidad de la relación, es decir, especifica cuántas instancias de una clase están asociadas a una instancia de la otra clase. Los tipos de multiplicidad son: Uno a uno, uno a muchos y muchos a muchos.
- **Herencia**: las jerarquías de generalización/especialización se conocen como herencia. Herencia es el mecanismo que permite a una clase de objetos incorporar atributos y métodos de otra clase, añadiéndolos a los que ya posee. Con la herencia se refleja una relación “**es_un**” entre clases. La clase de la cual se hereda se denomina superclase, y la que hereda subclase.

La generalización define una superclase a partir de otras. Por ejemplo, de las clases profesor y estudiante se obtiene la superclase persona. La especialización o especificación es la operación inversa, y en ella una clase se descompone en una o varias subclases. Ejemplo: de la clase empleado se pueden obtener las subclases secretaria, técnico e ingeniero.
 - **Agregación**: es un tipo de relación jerárquica entre un objeto que representa la totalidad de ese objeto y las partes que lo componen. Permite el agrupamiento físico de estructuras relacionadas lógicamente. Los objetos “**son-parte-de**” otro objeto completo. Ejemplo: motor, ruedas, carrocería son parte de automóvil.
 - **Composición**: es una forma de agregación donde la relación de propiedad es más fuerte, e incluso coinciden los tiempos de vida del objeto completo y las partes que lo componen. Ejemplo: en un sistema de Máquina de café, las relaciones entre la clase máquina y producto, o entre máquina y depósito de monedas, son de composición.
 - **Dependencia**: una relación de dependencia se utiliza entre 2 clases o entre una clase y una interfaz, e indica que una clase requiere de otra para proporcionar alguno de sus servicios.
- **Interfaces**: una interfaz es una especificación de la semántica de un conjunto de operaciones de una clase o paquete que son visibles desde otras clases o paquetes. Normalmente, se corresponde con una parte del comportamiento del elemento que la proporciona.
 - **Paquetes**: los paquetes se usan para dividir el modelo de clases del sistema de información, agrupando clases u otros paquetes según los criterios que sean oportunos. Las dependencias entre ellos se definen a partir de las relaciones establecidas entre los distintos elementos que se agrupan en estos paquetes.

Notación

Clases: una clase se representa como una caja, separada en 3 zonas por líneas horizontales.

En la zona superior se muestra el nombre de la clase y propiedades generales como el estereotipo. El nombre de la clase aparece centrado y si la clase es abstracta se representa en cursiva. El estereotipo, si se muestra, se sitúa sobre el nombre y entre el símbolo: << .. >>.

La zona central contiene una lista de atributos, uno en cada línea. La notación utilizada para representarlos incluye, dependiendo del detalle, el nombre del atributo, su tipo y su valor por defecto, con el formato:

```
visibilidad nombre : tipo = valor-inicial { propiedades }
```

La visibilidad será en general publica (+), privada (-) o protegida (#), aunque puede haber otros tipos de visibilidad dependiendo del lenguaje de programación empleado.

En la zona inferior se incluye una lista con las operaciones que proporciona la clase. Cada operación aparece en una línea con formato:

```
visibilidad nombre (lista-de-parámetros): tipo-devuelto { propiedad }
```

La visibilidad será en general publica (+), privada (-) o protegida (#), aunque como con los atributos, puede haber otros tipos de visibilidad dependiendo del lenguaje de programación. La lista de parámetros es una lista con los parámetros recibidos en la operación separados por comas. El formato de un parámetro es:

```
nombre : tipo = valor-por-defecto
```

La notación especificada se puede simplificar según el nivel de detalle con el que se quiera trabajar en un momento dado.

<< GUI >>
Formulario de Reservas
+ titulo : Titulo
+ prestatario: Informacion_prestatario
+ botonBuscarTitulo_Pulsado ()
+ botonBuscarPrestatario_Pulsado ()
+ botonOk_Pulsado ()
+ botonCancelar_Pulsado ()
+ tituloResultado ()
+ prestatarioResultado ()
- comprobarEstado ()
+ FormularioDeReservas ()
botonEliminarTitulo ()

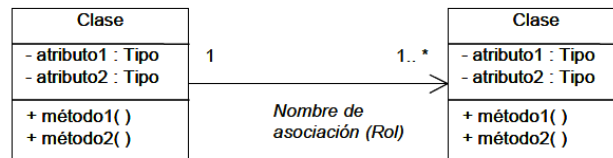
Relaciones: una relación de asociación se representa como una línea continua entre las clases asociadas. En una relación de asociación, ambos extremos de la línea pueden conectar con la misma clase, indicando que una instancia de una clase está asociada a otras instancias de la misma clase, lo que se conoce como asociación reflexiva.

La relación puede tener un nombre y un estereotipo, que se colocan junto a la línea. El nombre suele corresponderse con expresiones verbales presentes en las especificaciones, y define la semántica de la asociación. Los estereotipos permiten clasificar las relaciones en familias y se escribirán entre el símbolo: << ... >>.

Las diferentes **propiedades de la relación** se pueden representar con la siguiente notación:

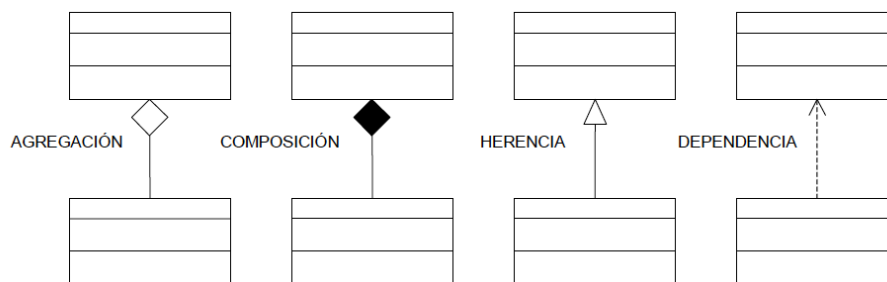
- Multiplicidad: puede ser un número concreto, un rango o una colección de números. La letra 'n' y el símbolo '*' representan cualquier número.
- Orden: se puede especificar si las instancias guardan un orden con la palabra clave '{ordered}'. Si el modelo es suficientemente detallado, se puede incluir una restricción que indique el criterio de ordenación.
- Navegabilidad: la navegación desde una clase a la otra se representa poniendo una flecha sin relleno en el extremo de la línea, indicando el sentido de la navegación.

- **Rol o nombre de la asociación:** este nombre se coloca junto al extremo de la línea que esta unida a una clase, para expresar cómo esa clase hace uso de la otra clase con la que mantiene la asociación.



Además, existen notaciones específicas para los otros tipos de relación, como son:

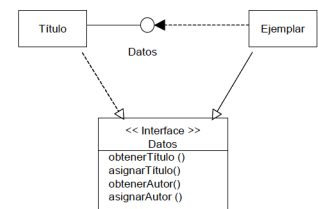
- **Agregación:** se representa con un rombo hueco en la clase cuya instancia es una agregación de las instancias de la otra.
- **Composición:** se representa con un rombo lleno en la clase cuya instancia contiene las instancias de la otra clase.
- **Dependencia:** una línea discontinua con una flecha apuntando a la clase cliente. La relación puede tener un estereotipo que se coloca junto a la línea, y entre el símbolo: << ... >>.
- **Herencia:** esta relación se representa como una línea continua con una flecha hueca en el extremo que apunta a la superclase.



Interfaces: una interfaz se representa como una caja con compartimentos, igual que las clases. En la zona superior se incluye el nombre y el estereotipo <<Interface>>. La lista de operaciones se coloca en la zona inferior, igual que en las representaciones de clases. La zona en la que se listan los atributos estará vacía o puede omitirse.

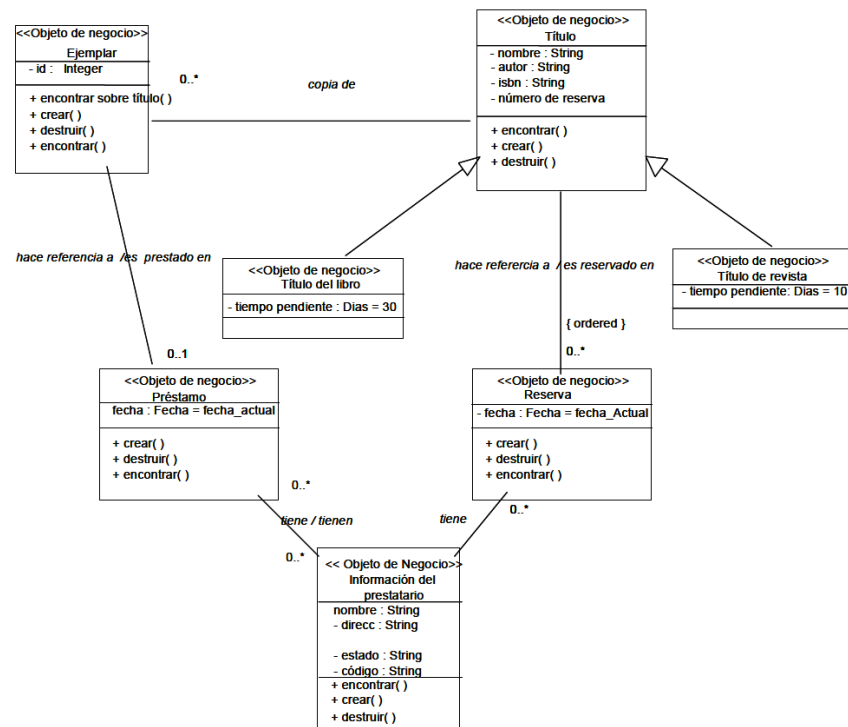
Existe una representación más simple para la interfaz: un círculo pequeño asociado a una clase con el nombre de la interfaz debajo. Las operaciones de la interfaz no aparecen en esta representación; si se quiere que aparezcan, debe usarse la primera notación.

Entre una clase que implementa las operaciones que una interfaz ofrece y esa interfaz se establece una relación de realización que, dependiendo de la notación elegida, se representará con una línea continua entre ellas cuando la interfaz se representa como un círculo y con una flecha hueca discontinua apuntando a la interfaz cuando se represente como una clase.

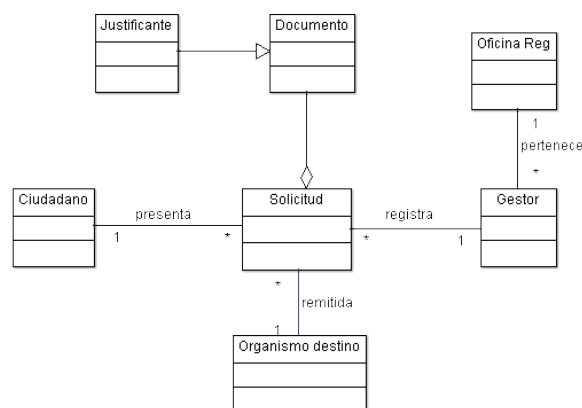


Paquetes: se representan mediante un icono con forma de carpeta y las dependencias con flechas discontinuas entre los paquetes dependientes.

Ejemplo: gestión de préstamos y reservas de libros y revistas de una biblioteca.



Ejemplo: gestión de Oficinas de Registro.



2

Diagrama de componentes

El diagrama de componentes proporciona una visión física de la construcción del sistema de información. Muestra la organización de los componentes software, sus interfaces y las dependencias entre ellos.

Como ya se ha indicado, los **elementos** de estos diagramas son los componentes software y las dependencias entre ellos.

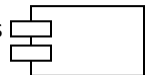
Un componente es un módulo de software que puede ser código fuente, código binario, un ejecutable, o una librería con una interfaz definida. Una interfaz establece las operaciones externas de un componente, las cuales determinan una parte del comportamiento del mismo.

Además se representan las dependencias entre componentes o entre un componente y la interfaz de otro, es decir uno de ellos usa los servicios o facilidades del otro.

Estos diagramas pueden incluir paquetes que permiten organizar la construcción del sistema de información en subsistemas y que recogen aspectos prácticos relacionados con la secuencia de compilación entre componentes, la agrupación de elementos en librerías, etc.

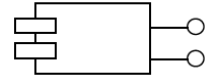
Notación

Componente: se representa como un rectángulo, con 2 pequeños rectángulos superpuestos perpendicularmente en el lado izquierdo.



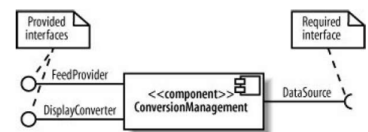
Para distinguir distintos tipos de componentes se les puede asignar un estereotipo, cuyo nombre estará dentro del símbolo: << ... >>

Interfaz: se representa como un pequeño círculo situado junto al componente que lo implementa y unido a él por una línea continua. La interfaz puede tener un nombre que se escribe junto al círculo. Un componente puede proporcionar más de una interfaz. La relación entre un componente y su interfaz se llama **realización**.



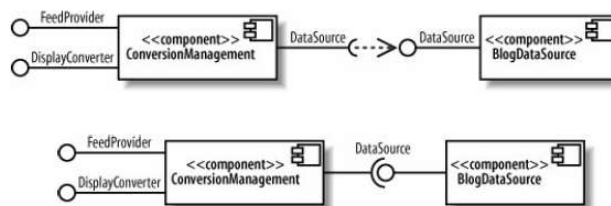
Podemos distinguir 2 tipos de interfaces:

- Interfaz de importación o REQUERIDA: un componente usa los servicios de otro.
- Interfaz de exportación o PROPORCIONADA: un componente realiza una interfaz (se ha definido junto al concepto de realización).

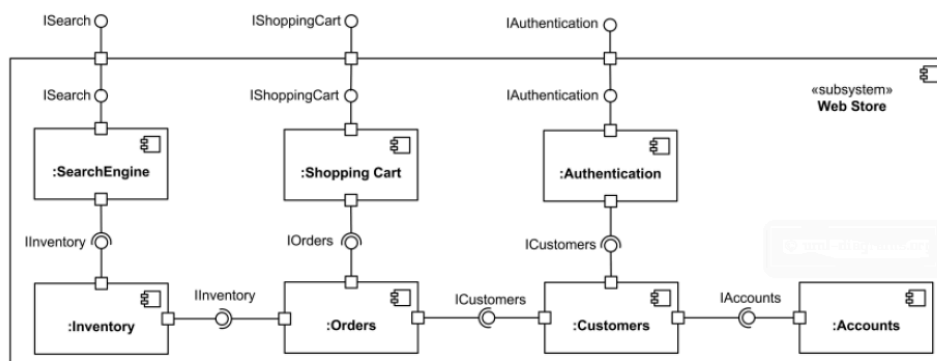


Paquete: se representa con un icono de carpeta.

Relación de dependencia: si un componente requiere una interfaz, necesita que otro componente implemente o realice.



Ejemplo: gestión web tienda online.



3 Diagrama de objetos

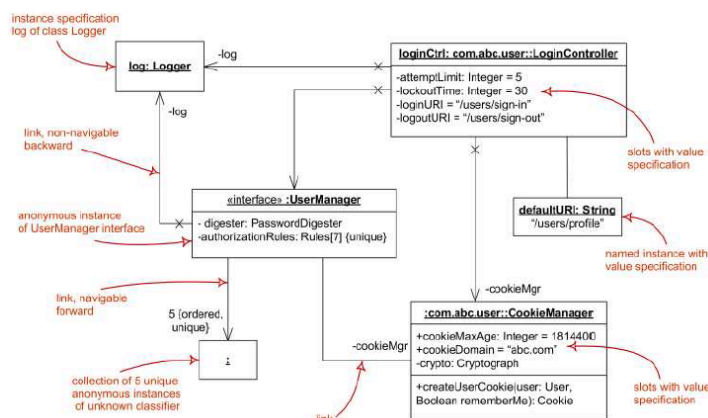
Contiene los objetos con sus valores. Muestra una instantánea del sistema en un determinado momento en el tiempo.

Notación

Se utiliza la misma notación que para el diagrama de clases. Se amplía con el concepto de objeto.

instancename :
ClassName

Ejemplo:



4 Diagrama de estructura compuesta

Un diagrama de estructura compuesta muestra la estructura interna de una clase (visión de la estructura como una caja blanca), incluyendo sus puntos de interacción con otras partes del sistema (comunicación entre estructuras).

Este tipo de diagrama es un complemento al diagrama de clases. Es usado cuando es necesario representar la relación inusual entre elementos.

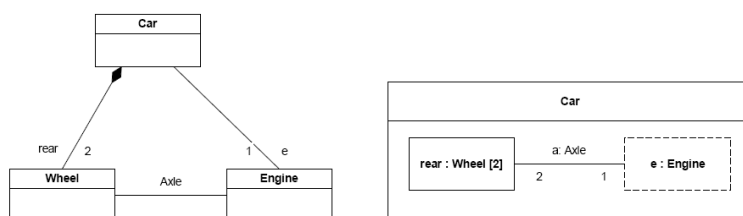
Los elementos de un diagrama de estructura compuesta son:

- **Parte o propiedad:** es un elemento de la estructura interna de una clase, representando una instancia o colección de instancias en tiempo de ejecución. La multiplicidad indica el número de instancias.
- **Conector:** comunica las partes de una estructura, conectando instancias concretas. La multiplicidad del conector indica el número de instancias que se conectan a cada lado.
- **Puerto:** indica un punto de interacción entre la clase y su entorno, especificando los servicios que provee y que requiere. Por eso, un puerto agrupa un conjunto de interfaces (requeridas y/o proporcionadas) relacionadas.
- **Colaboración:** define un conjunto de roles que cooperan entre sí para mostrar una funcionalidad concreta. Así, un rol es el papel que una instancia desempeña.

Notación

Clase: como ya se ha definido anteriormente.

Parte: es un objeto que se ubica dentro de una clase y que puede estar relacionado con otros objetos.

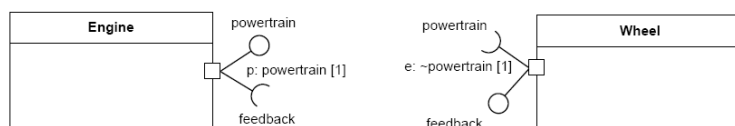


La multiplicidad de las partes se expresa mediante el nº de instancias concretas o un rango.

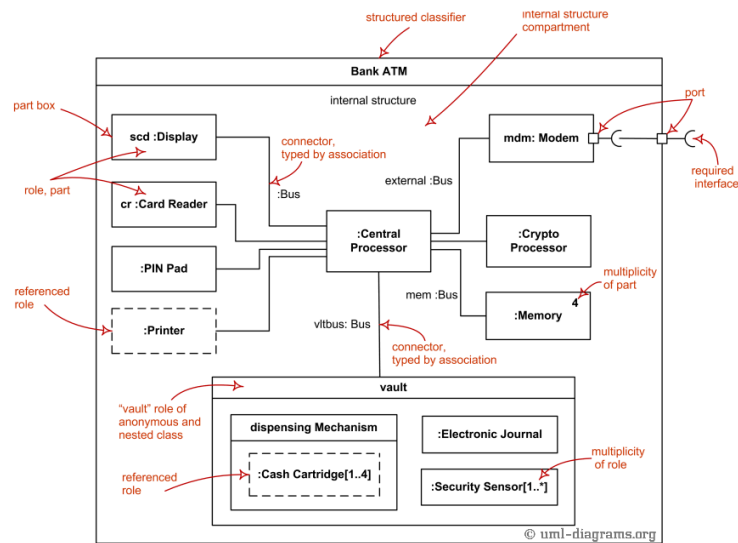
Ejemplo: 4 ruedas y 1 o 2 motores.



Puerto: se muestra gráficamente como un rectángulo con nombre. Las interfaces proporcionadas se representan con un círculo y las requeridas con un semicírculo.



Ejemplo:



5

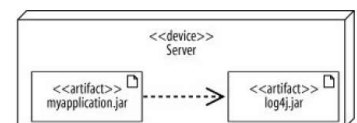
Diagrama de despliegue

El objetivo de estos diagramas es mostrar la disposición de las particiones físicas del sistema de información y la asignación de los componentes software a estas particiones. Es decir, las relaciones físicas entre los componentes software y hardware en el sistema a entregar.

En estos diagramas se representan dos tipos de elementos, **nodos** y **conexiones**, así como la distribución de componentes del sistema de información con respecto a la partición física del sistema.

Se propone una definición concreta de nodo, prescindiendo de determinados detalles, pero permitiendo una continuidad tanto en el diseño como en la construcción del sistema de información. Con este fin, se utiliza el nodo como partición física o funcional real, pero sin descender a detalles de infraestructura o dimensionamiento; por ejemplo, interesa si el nodo procesador es arquitectura Intel, pero no tanto si tiene dos o cuatro procesadores.

Dentro de un nodo se ubican los artefactos. Un **artefacto** es una pieza física de información que representa un software, como ejecutables, librerías, ficheros fuente, ficheros de configuración o BDs.



Las **conexiones** representan las formas de comunicación entre nodos.

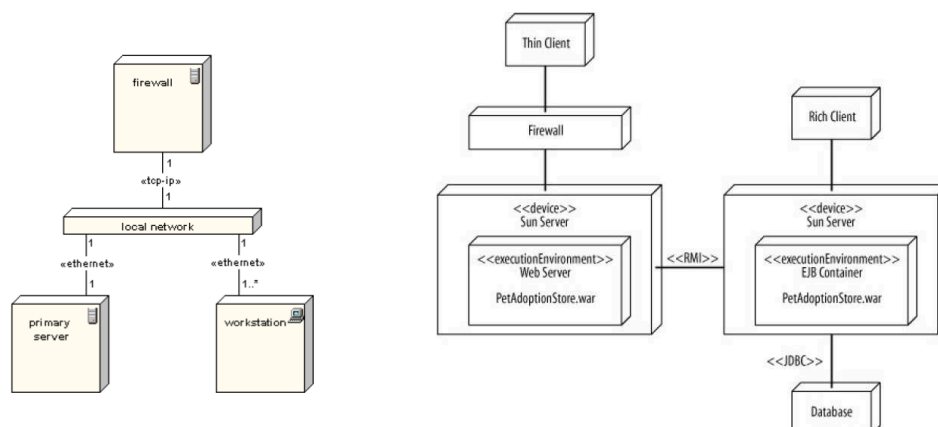
Además, a cada nodo se le asocia un subsistema de construcción que agrupa componentes software, permitiendo de este modo, determinar la distribución de estos componentes. Por lo tanto, un diagrama de despliegue puede incluir, dependiendo del nivel de detalle, todos los elementos descritos en la técnica de diagrama de componentes, además los nodos y las conexiones propios de esta técnica.

Notación

Nodo: se representa con la figura de un cubo. El nodo se etiqueta con un nombre representativo de la partición física que simboliza. Se pueden asociar a los nodos subsistemas de construcción.

Conexión: las conexiones se representan con una línea continua que une ambos nodos y pueden tener una etiqueta que indique el tipo de conexión. Ejemplo: canal, red, protocolo, etc. Se le puede asignar un estereotipo a la comunicación indicando el protocolo concreto.

Ejemplos:



6

Diagrama de paquetes

El objetivo de estos diagramas es obtener una visión más clara del sistema de información orientado a objetos, organizándolo en subsistemas, agrupando los elementos del análisis, diseño o construcción y detallando las relaciones de dependencia entre ellos. El mecanismo de agrupación se denomina Paquete.

Estrictamente hablando, los paquetes y sus dependencias son elementos de los diagramas de casos de uso, de clases y de componentes, por lo que se podría decir que el diagrama de paquetes es una extensión de éstos.

Estos diagramas contienen 2 tipos de elementos:

- **Paquetes:** un paquete es una agrupación de elementos, bien sea casos de uso, clases o componentes. Los paquetes pueden contener a su vez otros paquetes anidados que en última instancia contendrán alguno de los elementos anteriores.
- **Dependencias entre paquetes:** existe una dependencia cuando un elemento de un paquete requiere de otro que pertenece a un paquete distinto. Es importante resaltar que las dependencias no son transitivas.

Se pueden optimizar estos diagramas teniendo en cuenta cuestiones como: la generalización de paquetes, el evitar ciclos en la estructura del diagrama, la minimización de las dependencias entre paquetes, etc.

Resumiendo, el diagrama de paquetes es un mecanismo para gestionar la complejidad del modelado, permitiendo organizar los elementos del modelo y controlar el acceso a sus contenidos (visibilidad).

Notación

Paquete: se representa mediante un símbolo con forma de ‘carpeta’ en el que se coloca el nombre en la pestaña y el contenido del paquete dentro de la ‘carpeta’. En los casos en que no sea visible el contenido del paquete se podrá colocar en su lugar el nombre.

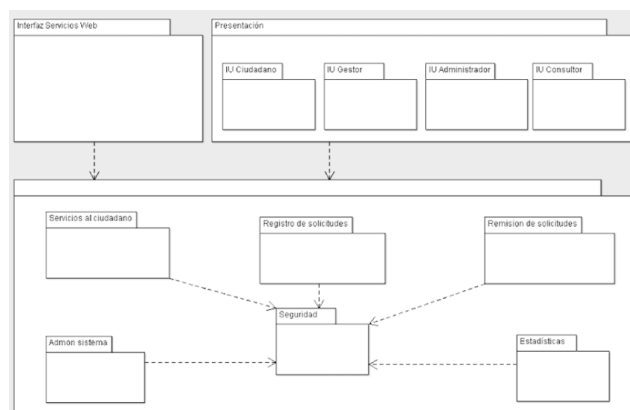
Si el paquete tiene definido un estereotipo, éste se representa encima del nombre entre el símbolo << ... >>, y si se definen propiedades, se representan debajo del nombre y entre llaves.

La visibilidad de los elementos que forman el paquete se debe indicar anteponiendo a su nombre los símbolos:

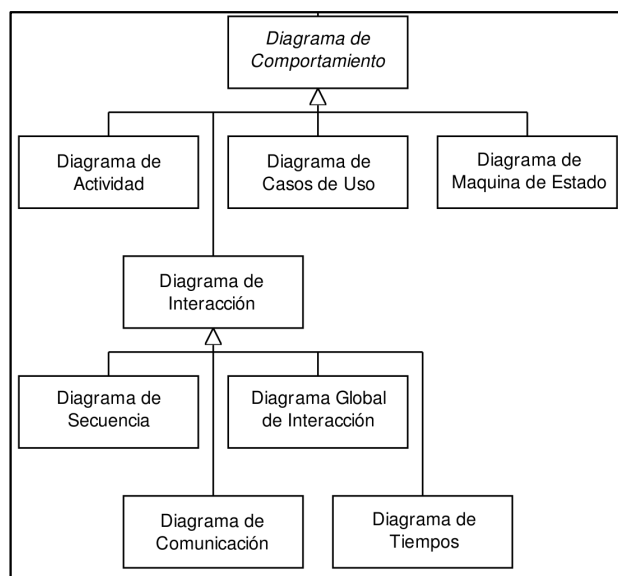
- ‘+’ para los públicos.
- ‘-’ para los privados.
- ‘#’ para los protegidos.

Dependencia: Las dependencias se representan con una flecha discontinua con inicio en el paquete que depende del otro.

Ejemplo: Diagrama de paquetes de un sistema de Registro.



2. Diagrama de comportamiento (vista dinámica)



1

Diagrama de actividad

El Diagrama de Actividades es una variante de los Diagramas de Estados, organizado con respecto a las acciones. Muestra un flujo de actividades.

Se utilizan:

- En el modelado de los procesos del negocio: permiten especificar y evaluar el flujo de trabajo de los procesos de negocio.
- En el análisis de un caso de uso: permiten comprender qué acciones deben ocurrir y cuáles son las dependencias de comportamiento.
- En la comprensión del flujo de trabajo, a través de varios casos de uso: permiten representar con claridad las relaciones de flujo de trabajo (workflow) entre varios casos de uso.
- Generalmente se suelen utilizar para modelar los pasos de un algoritmo.

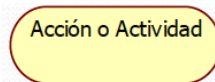
Los elementos de este diagrama son:

- **Acción:** es la ejecución de una acción atómica. Por lo que ni se descompone y la aparición de eventos interrumpe su ejecución. Se considera que su duración es insignificante.
- **Actividad:** no es atómica, se compone de acciones y actividades y tiene duración. Las actividades se enlazan entre sí por transiciones automáticas.
- **Transición:** cambio de estado producido por un evento. Existen varios tipos:

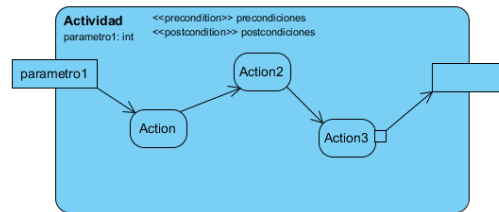
- Secuencial: al finalizar una acción, el control continúa por la transición hasta el siguiente estado.
- Decisión (decisión node): transición múltiple que sale de un estado y que en función de una condición sigue un camino.
- Fusión (merge node): unen varios flujos de ejecución.
- Bifurcación (fork node): permite expresar paralelismo. Las actividades invocadas después de una bifurcación son concurrentes.
- Unión (join node): los flujos entrantes se sincronizan, esto es, esperan hasta que todos los flujos de entrada han alcanzado la unión.

Notación

Acciones y actividades tienen la misma representación gráfica.

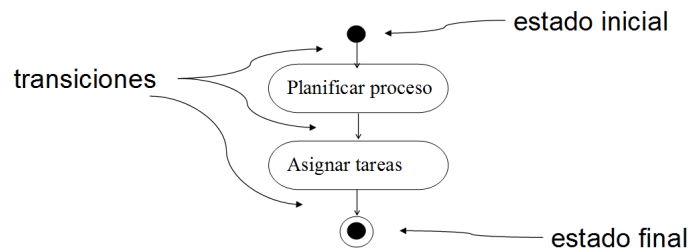


La actividad tiene nombre, parámetros y precondiciones y postcondiciones.

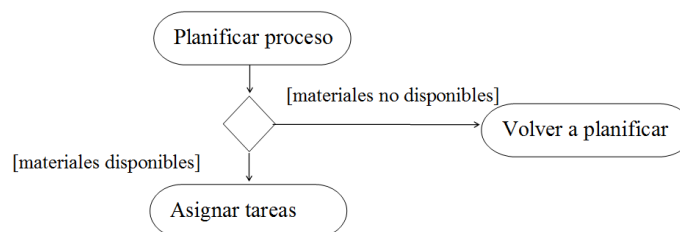


Transición: según el tipo, tenemos:

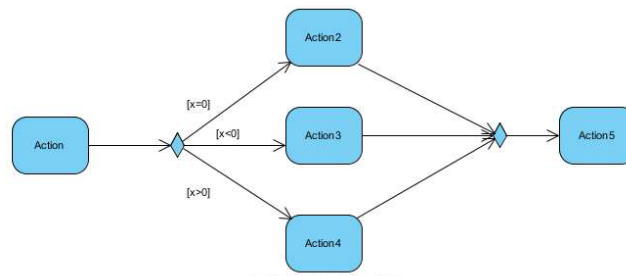
- Secuencial: se representa por una flecha dirigida.



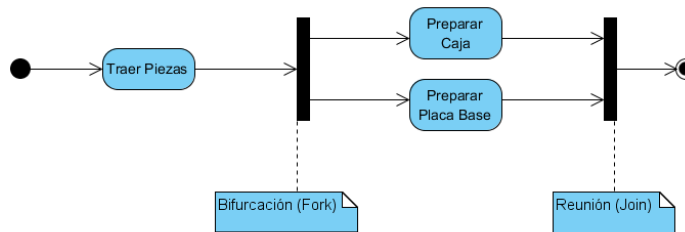
- Decisión: se representa mediante un rombo al cual llega la transición del estado origen y del cual salen múltiples transiciones a los estados destino.



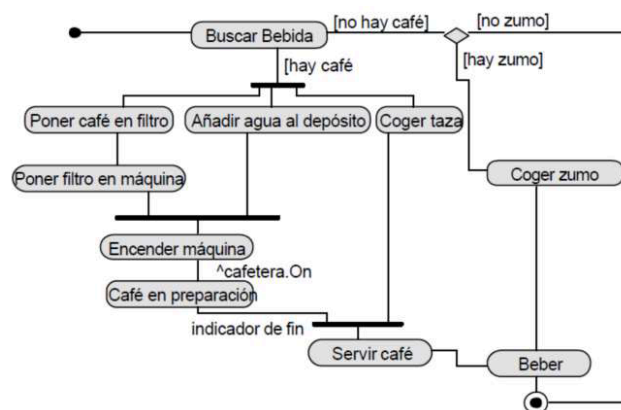
- Fusión: representado mediante un rombo como punto de llegada de varios flujos.



- Bifurcación: se utiliza un rectángulo vertical del cual salen tantas transiciones secuenciales como “hilos” en paralelo se requieran.
- Reunión: representado mediante un rectángulo vertical al cual llegan transiciones concurrentes.



Ejemplo: máquina de café.



2

Diagrama de casos de uso

Los objetivos de los casos de uso son los siguientes:

- Capturar los requisitos funcionales del sistema y expresarlos desde el punto de vista del usuario.
- Guiar todo el proceso de desarrollo del sistema de información.

Los casos de uso proporcionan, por tanto, un modo claro y preciso de comunicación entre cliente y desarrollador. Desde el punto de vista del cliente proporcionan una visión de “caja negra” del sistema, esto es, cómo aparece el sistema desde el exterior sin necesidad de entrar en los detalles de su construcción. Para los desarrolladores, suponen el punto de partida y el eje sobre el que se apoya todo el desarrollo del sistema en sus procesos de análisis y diseño.

Un caso de uso (CU) es una secuencia de acciones realizadas por el sistema, que producen un resultado observable y valioso para un usuario en particular, es decir, representa el comportamiento del sistema con el fin de dar respuestas a los usuarios. Aquellos casos de uso que resulten demasiado complejos se pueden descomponer en un segundo nivel, en el que los nuevos casos de uso que intervengan resulten más sencillos y manejables.

Para especificar este comportamiento existen una serie de recomendaciones o técnicas que se aplican dependiendo del momento del desarrollo que se esté y de la complejidad del CU. Puede ser desde una simple descripción textual que recoja un requisito funcional a una especificación del CU, e incluso un conjunto de diagramas.

Un CU recoge, en un primer momento, una descripción general. Esta descripción reflejará posiblemente uno o varios requisitos funcionales del sistema o formará parte de algún requisito.

Se puede completar la descripción definiendo cuáles son las precondiciones y postcondiciones del sistema, es decir, qué condiciones deben cumplirse para que se realice un CU y cuáles son aquellas condiciones que se cumplen posteriormente al CU.

También se pueden enumerar los diferentes escenarios del CU si los tuviese y dar una breve descripción de ellos. Los escenarios son los distintos caminos por los que puede evolucionar un CU, dependiendo de las condiciones que se van dando en su realización.

Estos diagramas presentan 2 tipos de **elementos** fundamentales:

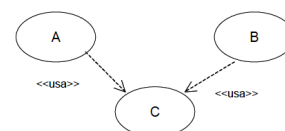
- **Actores.** Un actor es algo o alguien que se encuentra fuera del sistema y que interactúa con él. En general, los actores serán los usuarios del sistema y los sistemas externos al que se esté desarrollando. Si se habla de usuarios, un actor es el papel que puede llevar a cabo en cuanto a su forma de interactuar con el sistema, es decir, un único actor puede representar a muchos usuarios diferentes y de la misma forma, un usuario puede actuar como actores diferentes.
- **Casos de uso.** Un CU representa el comportamiento que ofrece el sistema de información desde el punto de vista del usuario. Típicamente será un conjunto de transacciones ejecutadas entre el sistema y los actores. Para facilitar la comprensión de los CU del sistema de información en el análisis, es posible agruparlos en paquetes según funcionalidades semejantes o relacionadas.

Además de estos elementos, un diagrama de CU presenta relaciones. Las relaciones pueden tener lugar entre actores y casos de uso o entre casos de uso.

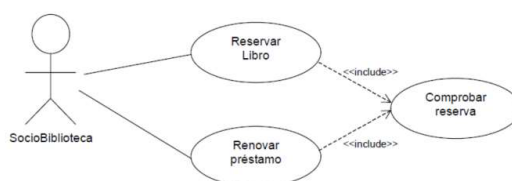
La relación entre un actor y un CU es una relación de comunicación, que indica que un actor interviene en el CU. Normalmente, el actor aporta información para la realización de un CU o recibe información como resultado de la realización del mismo, por ello, esta relación puede ser unidireccional o bidireccional, aunque generalmente se muestra como bidireccional, ya que no es necesario especificar en detalle estas relaciones.

La relación entre CU es una relación unidireccional. Esta relación puede presentar uno de los 2 siguientes tipos: “usa” y “extiende”.

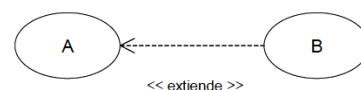
- La **relación “usa”** se utiliza cuando se quiere reflejar un comportamiento COMÚN en varios CU. Es decir, si los CU A y B presentan una parte común, ésta se puede sacar a un tercer caso de uso C. Entonces, habrá una relación “usa” del CU A al C y otra del B al C. La relación se puede también estereotipar mediante “include”.



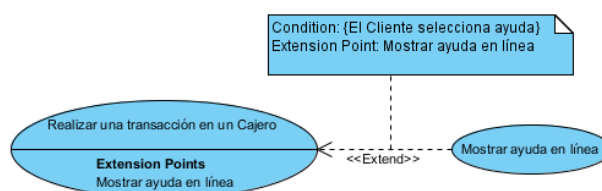
Ejemplo:



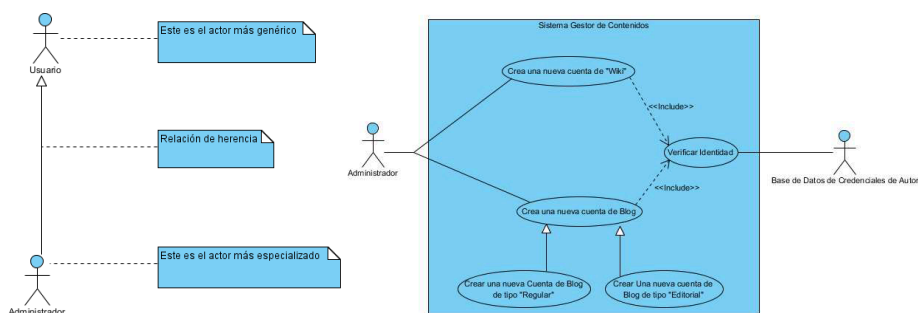
- La **relación “extiende”** se utiliza cuando se quiere reflejar un comportamiento OPCIONAL de un CU. Por ejemplo, se tiene el CU A que representa un comportamiento habitual del sistema. Sin embargo, dependiendo de algún factor (denominado punto de extensión), este CU puede presentar un comportamiento adicional o ligeramente diferente, que se podría reflejar en un CU B. En este caso, habrá una relación “extiende” del CU B al A. También se puede estereotipar mediante “extend”.



Ejemplo:

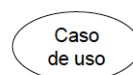


Por último, podemos hacer uso de la generalización y especialización para modelar relaciones entre los actores o entre CU, permitiendo al concepto especializado heredar las características del concepto del que hereda.

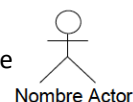
Ejemplos:Notación

El diagrama de CU es un grafo de actores, CU y las relaciones entre estos elementos. Opcionalmente, los CU se pueden enmarcar en un cuadrado que representa los límites del sistema.

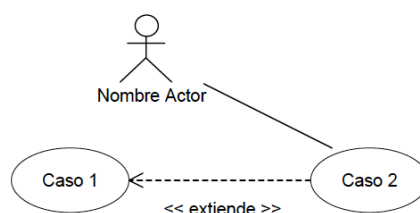
Caso de Uso: se representa mediante una elipse con el nombre del CU dentro o debajo.



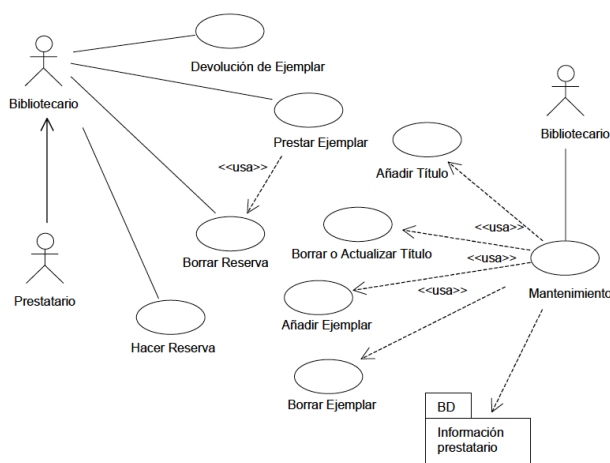
Actor: se representa con una figura de 'hombre de palo' con el nombre del actor debajo de la figura.



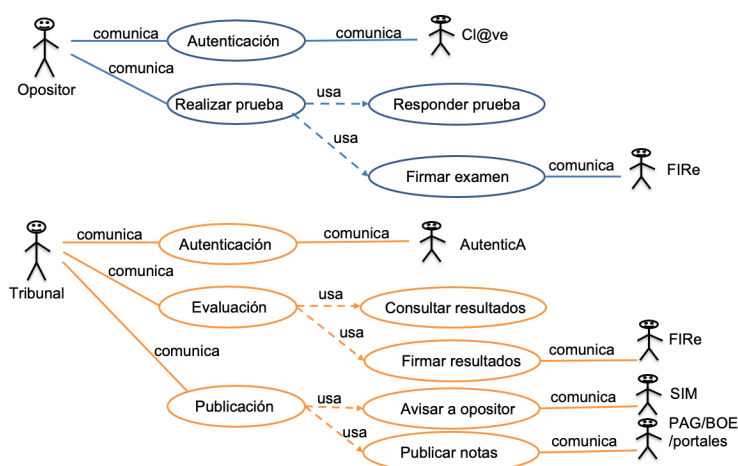
Relación: dependiendo del tipo de relación, la representación en los diagramas será distinta. Así pues, las relaciones entre un actor y un CU se representan mediante una línea continua entre ellos. Las relaciones entre CU se representan con una flecha discontinua con el nombre del tipo de relación como etiqueta. En las relaciones "extensión" la flecha parte del CU con el comportamiento adicional hacia aquel que recoge el comportamiento básico y en las relaciones "usa" desde el CU básico hacia el que representa el comportamiento común.



Ejemplo: gestión de los préstamos y reservas de libros y revistas en una biblioteca.



Ejemplo: sistema de gestión de procesos selectivos.



3 Diagrama de máquina de estado

En UML 1.X se denominaba Diagrama de transición de estados.

Un diagrama de máquina de estado muestra el comportamiento dependiente del tiempo de un sistema de información. Representa los **estados** que puede tomar un componente o un sistema y muestra los **eventos** que implican el cambio de un estado a otro.

Los 2 elementos principales en estos diagramas son los estados y las posibles transiciones entre ellos:

- El **estado** de un componente o sistema representa algún comportamiento que es observable externamente y que perdura durante un periodo de tiempo finito. Viene dado por el valor de uno o varios atributos que lo caracterizan en un momento dado.
- Una **transición** es un cambio de estado producido por un evento y refleja los posibles caminos para llegar a un estado final desde un estado inicial. Así pues, una transición de un estado A a un estado B, se produce cuando se origina el evento asociado y se satisface la condición especificada, en cuyo caso se ejecuta la acción de salida de A, la acción de entrada a B y la acción asociada a la transición. Dentro las transiciones, tienen relevancia los **pseudo-estados**, utilizados para expresar caminos de transición complejos (ver diagrama de actividades): decisión, unión, bifurcación, reunión...

Desde un estado pueden surgir varias transiciones en función del evento que desencadena el cambio de estado, teniendo en cuenta que, las transiciones que provienen del mismo estado no pueden tener el mismo evento, salvo que exista alguna condición que se aplique al evento.

Un sistema sólo puede tener un estado inicial, que se representa mediante una transición sin etiquetar al primer estado normal del diagrama. Pueden existir varias transiciones desde el estado inicial, pero deben tener asociadas condiciones, de manera que sólo una de ellas

sea la responsable de iniciar el flujo. En ningún caso puede haber una transición dirigida al estado inicial.

El estado final representa que un componente ha dejado de tener cualquier interacción o actividad. No se permiten transiciones que partan del estado final. Puede haber varios estados finales en un diagrama, ya que es posible concluir el ciclo de vida de un componente desde distintos estados y mediante diferentes eventos, pero dichos estados son mutuamente excluyentes, es decir, sólo uno de ellos puede ocurrir durante una ejecución del sistema.

Los diagramas de máquina de estado comprenden además otros dos elementos que ayudan a clarificar el significado de los distintos estados por los que pasa un componente o sistema. Estos elementos se conocen como acciones y actividades:

- Una **acción** es una operación instantánea asociada a un evento, cuya duración se considera no significativa y que se puede ejecutar: dentro de un estado, al entrar en un estado o al salir del mismo.
- Una **actividad** es una operación asociada a un estado que se ejecuta durante un intervalo de tiempo hasta que se produce el cambio a otro estado.

Para aquellos estados que tengan un comportamiento complejo, se puede utilizar un diagrama de máquina de estado de más bajo nivel. Estos diagramas se pueden mostrar por separado o bien incluirse en el diagrama de más alto nivel, dentro del contorno del estado que representa. En cualquier caso, su contenido formará un contexto independiente del resto, con sus propios estados inicial y final.

Notación

Estado: un estado se representa como un rectángulo con las esquinas redondeadas. El nombre del estado se coloca dentro del rectángulo y debe ser único en el diagrama. Si se repite algún nombre, se asume que simboliza el mismo estado.

Las acciones y actividades descritas como respuesta a eventos que no producen un cambio de estado se representan dentro del rectángulo con el formato:

nombre-evento (parámetros) [condición] /acción

El estado inicial se representa con un pequeño círculo relleno, y el estado final como un pequeño círculo relleno con una circunferencia que lo rodea.



estado inicial

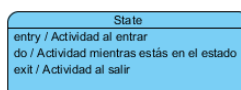


estado final

Transición: se representa con una flecha continua que une dos estados y que se dirige al estado al que cambia el componente. Junto a ella se coloca una etiqueta que debe contener al menos el nombre del evento que provoca la transición. Según el nivel de detalle, puede presentar otros elementos con el formato siguiente:

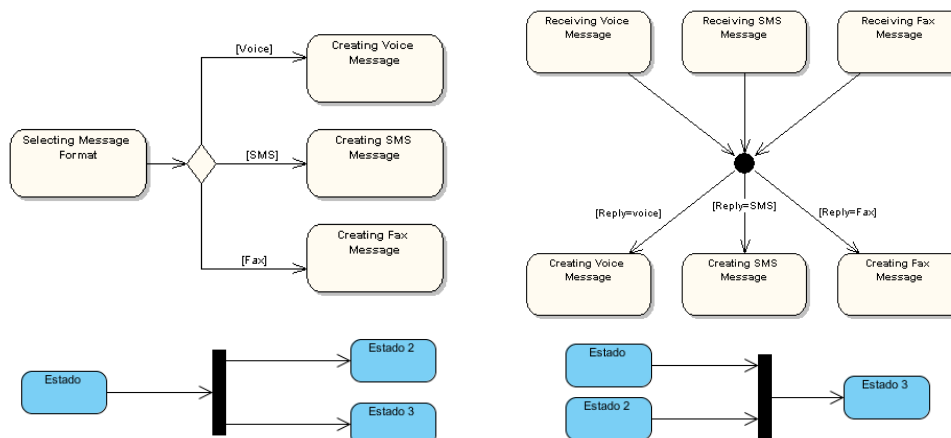
nombre-evento (parámetros) [condición] /acción

Acciones: se indican dentro del propio estado.



Pseudo-estados: se representan de la misma forma que las transiciones en el diagrama de actividades.

Ejemplos: decisión, unión, bifurcación y reunión respectivamente.



4 Diagrama de interacción

Incluye los diagramas de secuencia, comunicación, global de interacción y de tiempos.

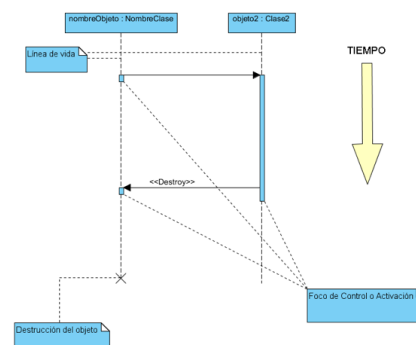
5 Diagrama de secuencia

El diagrama de secuencia es un tipo de diagrama de interacción cuyo objetivo es describir el **comportamiento dinámico** del sistema de información haciendo énfasis en la secuencia de los **mensajes** intercambiados por los objetos.

Un diagrama de secuencia tiene 2 dimensiones, el eje vertical representa el tiempo y el eje horizontal los diferentes objetos. El tiempo avanza desde la parte superior del diagrama hacia la inferior. Normalmente, en relación al tiempo sólo es importante la secuencia de los mensajes, sin embargo, en aplicaciones de tiempo real se podría introducir una escala en el eje vertical.

Respecto a los objetos, es irrelevante el orden en que se representan, aunque su colocación debería poseer la mayor claridad posible. Cada objeto tiene asociados una línea de vida y focos de control:

- La **línea de vida** indica el intervalo de tiempo durante el que existe ese objeto.
- Un **foco de control o activación** muestra el periodo de tiempo en el cual el objeto se encuentra ejecutando alguna operación, ya sea directamente o mediante un procedimiento concurrente.



Notación

Objeto y línea de vida: un objeto se representa como una línea vertical discontinua, llamada línea de vida, con un rectángulo de encabezado con el nombre del objeto en su interior. También se puede incluir a continuación el nombre de la clase, separando ambos por dos puntos.

Si el objeto es creado en el intervalo de tiempo representado en el diagrama, la línea comienza en el punto que representa ese instante y encima se coloca el objeto. Si el objeto es destruido durante la interacción que muestra el diagrama, la línea de vida termina en ese punto y se señala con un aspa de ancho equivalente al del foco de control.

En el caso de que un objeto existiese al principio de la interacción representada en el diagrama, dicho objeto se situará en la parte superior del diagrama, por encima del primer mensaje. Si un objeto no es eliminado en el tiempo que dura la interacción, su línea de vida se prolonga hasta la parte inferior del diagrama.

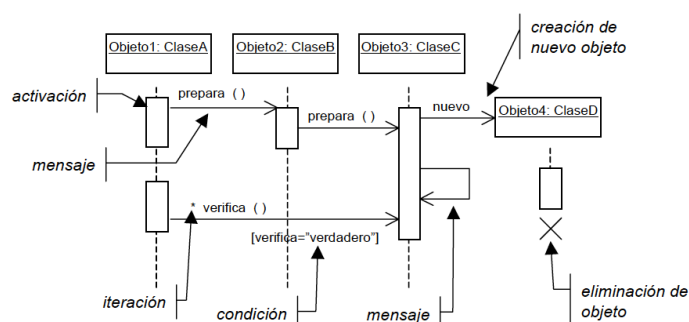
La línea de vida de un objeto puede desplegarse en 2 o más líneas para mostrar los diferentes flujos de mensajes que puede intercambiar un objeto, dependiendo de alguna condición.

Foco de control o activación: se representa como un rectángulo delgado superpuesto a la línea de vida del objeto. Su largo dependerá de la duración de la acción. La parte superior del rectángulo indica el inicio de una acción ejecutada por el objeto y la parte inferior su finalización.

Mensaje: se representa como una flecha horizontal entre las líneas de vida de los objetos que intercambian el mensaje. La flecha va desde el objeto que envía el mensaje al que lo recibe. Además, un objeto puede mandarse un mensaje a sí mismo, en este caso la flecha comienza y termina en su línea de vida.

Los mensajes pueden ser síncronos si se espera hasta obtener respuesta, de retorno o respuesta (línea discontinua) o asíncronos si el objeto origen no espera respuesta.

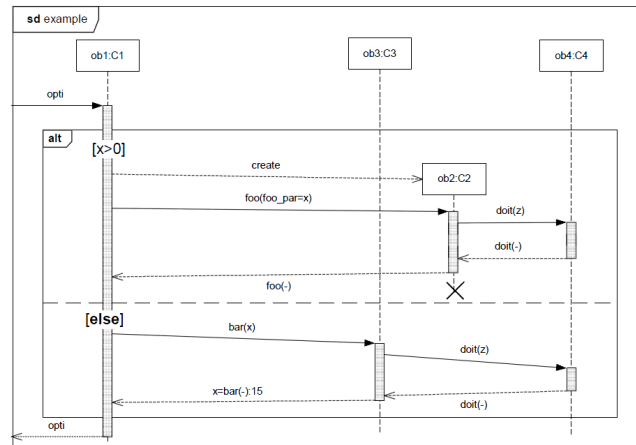
La flecha tiene asociada una etiqueta con el nombre del mensaje y los argumentos. También pueden ser etiquetados los mensajes con un número de secuencia, sin embargo, este número no es necesario porque la localización física de las flechas que representan a los mensajes ya indica el orden de los mismos.



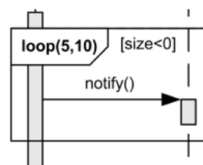
Los mensajes pueden presentar también condiciones e iteraciones. Una condición se representa mediante una expresión booleana encerrada entre corchetes junto a un mensaje, e indica que ese mensaje sólo es enviado en caso de ser cierta la condición. Una iteración se representa con un asterisco y una expresión entre corchetes, que indica el número de veces que se produce.

Fragmentos combinados: permiten agregar un grado de lógicas de procedimientos a los diagramas. Es una o más secuencias de procesos incluidas en un marco y ejecutadas bajo circunstancias nombradas específicas. De entre los fragmentos disponibles destacamos:

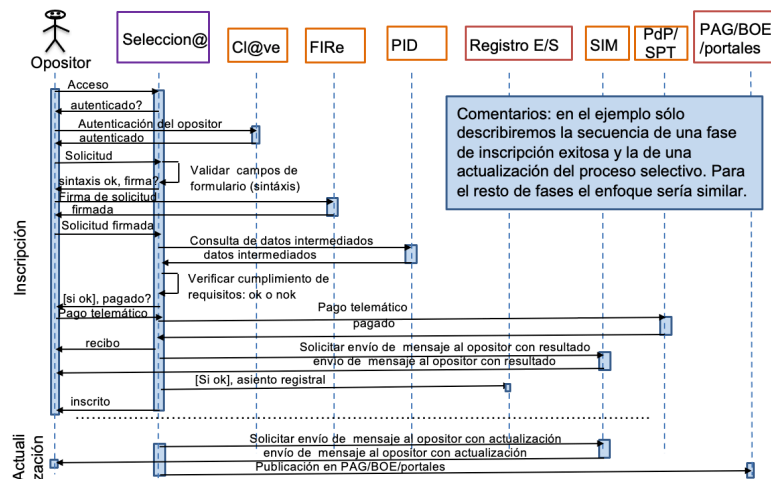
- El fragmento Alternative (“alt”) modela estructuras if-then-else.



- El fragmento Option (“opt”) modela estructuras condicionales.
- El fragmento Break modela una secuencia alternativa de eventos que se procesa en lugar de todo del resto del diagrama.
- El fragmento Parallel (“par”) modela procesos concurrentes.
- El fragmento Loop modela bucles.



Ejemplo: inscripción en un proceso selectivo y actualización del proceso selectivo.



6 Diagrama de comunicación

En UML 1.X se denominaba Diagrama de colaboración.

El diagrama de comunicación es un tipo de diagrama de interacción cuyo objetivo es describir el comportamiento dinámico del sistema de información mostrando **cómo interactúan los objetos entre sí**, es decir, con qué otros objetos tienen vínculos o intercambia mensajes un determinado objeto.

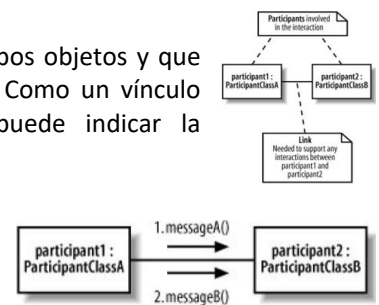
Un diagrama de comunicación muestra la misma información que un diagrama de secuencia pero de forma diferente. En los diagramas de comunicación no existe una secuencia temporal en el eje vertical; es decir, la colocación de los mensajes en el diagrama no indica cuál es el orden en el que se suceden. Además, la colocación de los objetos es más flexible y permite mostrar de forma más clara cuáles son las colaboraciones entre ellos. En estos diagramas la comunicación entre objetos se denomina vínculo o enlace (link) y estará particularizada mediante los mensajes que intercambian.

Notación

Objeto: se representa con un rectángulo dentro del que se incluye el nombre del objeto y, si se desea, el nombre de la clase, separando ambos por dos puntos.

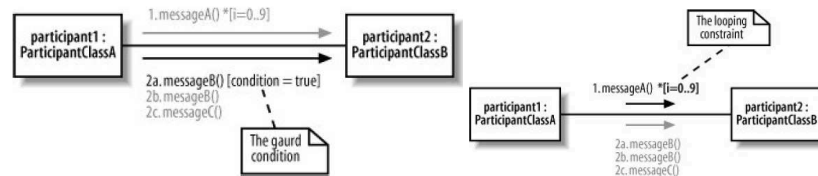
Vínculo o enlace: se representa como una línea continua que une ambos objetos y que puede tener uno o varios mensajes asociados en ambas direcciones. Como un vínculo instancia una relación de asociación entre clases, también se puede indicar la navegabilidad del mismo mediante una flecha.

Mensaje: se representa con una pequeña flecha colocada junto a la línea del vínculo al que está asociado. La dirección de la flecha va del objeto emisor del mensaje al receptor del mismo. Junto a ella, se coloca el nombre del mensaje y sus argumentos.

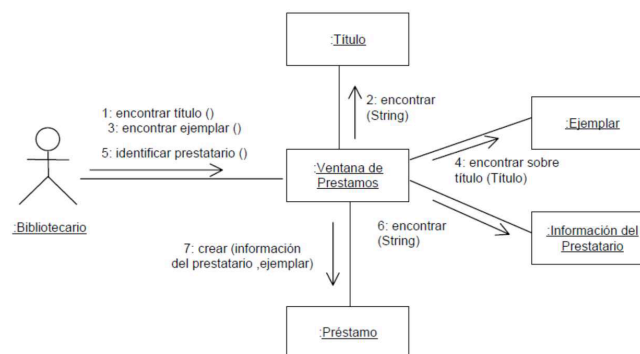


A diferencia de los diagramas de secuencia, en los diagramas de comunicación siempre se muestra el número de secuencia del mensaje delante de su nombre, ya que no hay otra forma de conocer la secuencia de los mismos.

Además, los mensajes pueden tener asociadas condiciones e iteraciones que se representarán como en los diagramas de secuencia.



Ejemplo: diagrama de comunicación para el caso de uso Prestar un ejemplar de un sistema de préstamos de una biblioteca.



7 Diagrama global de interacción

Representa una vista global sobre el resto de diagramas de interacción (secuencia, comunicación y tiempo).

Tiene la misma estructura que un diagrama de actividad (es una especialización de ellos), salvo que en vez actividades se representan interacciones. Tienen un nodo de comienzo y otro de finalización y los pseudo-estados nodos decisión, fusión, bifurcación y reunión.

Otros elementos:

- **Interacción:** dentro se representan los distintos diagramas de interacción. Pueden llevar el nombre en la parte superior izquierda.
- **Uso de Interacción:** indica una referencia a una interacción.

Notación

Interacción:



Uso de Interacción:



3. Esquema diagramas de UML

ESTRUCTURALES

- **Diagrama de Clases:** muestra la estructura estática del sistema, incluyendo clases, atributos, métodos y las relaciones entre los objetos.
- **Diagrama de Componentes:** visión física que muestra la organización y dependencia de los componentes software.
- **Diagrama de Objetos:** muestra una instantánea del sistema en un determinado momento en el tiempo.
- **Diagrama de Estructura Compuesta:** muestra la estructura interna de una clase y las colaboraciones que esta estructura hace posible.
- **Diagrama de Despliegue:** muestra la disposición de las particiones físicas del sistema de y la asignación de los componentes software a estas particiones.
- **Diagrama de Paquetes:** visión lógica del sistema que lo organiza en subsistemas y muestra las dependencias entre ellos.

COMPORTAMIENTO

- **Diagrama de Actividades:** muestra el flujo de actividades en un sistema (variante del diagrama de máquina de estado).
- **Diagrama de Casos de Uso:** captura los requisitos funcionales del sistema desde el punto de vista del usuario.
- **Diagrama de Máquina de Estado:** muestra el comportamiento dependiente del tiempo de un sistema.
- **Diagramas de Interacción:**
 - o **Diagrama de Secuencia:** describe el comportamiento dinámico del sistema haciendo énfasis en la secuencia mensajes intercambiados por los objetos.
 - o **Diagrama de Comunicación:** describe el comportamiento dinámico del sistema mostrando cómo interactúan los objetos entre sí.
 - o **Diagrama de Tiempo:** muestra el cambio en el estado o valor de uno o más objetos en el tiempo.
 - o **Diagrama Global de Interacción:** vista global sobre el resto de diagramas de interacción (secuencia, comunicación y tiempo).